

10. Host-untrusted (VM) software generates a full cacheline memory write request to the same test memory address but with a non-TEE opcode and known data pattern B.
11. Host software verifies that the write returned a TEE opcode in the response.
12. Host-untrusted (VM) software generates a memory read request to the same test memory address but with a non-TEE opcode.
13. Host software verifies that the read returned a TEE opcode in the response and all 1s for the data.
14. Host-trusted TEE (TVM) software generates a memory read request to the same test memory address but with a TEE opcode.
15. Host software verifies that the read returned a TEE opcode in the response and expected data pattern A.
16. Host software issues Set Target TE State to set TE State to 0 for the test memory address.
17. Host-untrusted (VM) software generates a full cacheline memory write request to the same test memory address but with a non-TEE opcode and known data pattern A.
18. Host software verifies that the write returned a non-TEE opcode in the response.
19. Host-untrusted (VM) software generates a memory read request to the same test memory address but with a non-TEE opcode.
20. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.
21. Host-trusted TEE (TVM) software generates a full cacheline memory write request to the same test memory address but with a TEE opcode and known data pattern B.
22. Host software verifies that the write returned a non-TEE opcode in the response.
23. Host-trusted TEE (TVM) software generates a memory read request to the same test memory address but with a TEE opcode.
24. Host software verifies that the read returned a non-TEE opcode in the response and all 1s for the data
25. Host-untrusted (VM) software generates a memory read request to the same test memory address but with a non-TEE opcode.
26. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.

Pass Criteria:

- TEE/non-TEE opcode returned is correct for all writes and reads
- Read data returned for all reads is as expected
- Data pattern of all 1s returned for reads that have a TE State mismatch

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Pass criteria is not met

14.11.7.8 Initiator-based Memory Encryption

This test verifies basic optional initiator-based memory encryption by disabling target-based encryption.

Prerequisites:

- Device must support CXL.cachemem TSP security
- Device must support Compliance Mode DOE and SPDM over DOE
- Host software has established a secure SPDM link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for Explicit Out-of-band TE State Changes, Read Access Control, and Write Access Control

Topologies:

- SHDA

Test Steps:

1. Host software issues Set Target Configuration to disable target encryption:
 - a. All Memory Encryption Features Enable flag bits are cleared to 0.
2. Host software receives Set Target Configuration Response.
3. Host software issues Lock Target Configuration to make the configuration immutable and to enable the ability to receive TEE opcodes.
4. Host software receives Lock Target Configuration Response.
5. Host software encrypts data before writing.
6. Host software writes encrypted data to the locked target device with a known data pattern.
7. Host software reads encrypted data from the target.
8. Host software decrypts the data and verifies that the data matches the known data pattern.

Pass Criteria:

- Data does not match the expected pattern after the data is decrypted by the host

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Pass criteria is not met

14.11.7.9 Target-based CKID-based Memory Encryption Invalid CKID Range

This test verifies basic invalid CKID range handling for optional target-based CKID-based memory encryption when the target supports a limited number of CKIDs. The following TSP-related table is tested:

- [Table 11-26, “Target Behavior for Invalid CKID Ranges”](#)

Prerequisites:

- Device must support CXL.cachemem TSP security
- Device must support Compliance Mode DOE and SPDM over DOE
- Host software has established a secure SPDM link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for target-based CKID-based encryption and the target limits the supported CKID range (CKID Base Required is set in [Test 14.11.7.3](#))

Topologies:

- SHDA

Test Steps:

- Host software issues Set Target Configuration to enable target-based CKID-based memory encryption:
 - Encryption bit and CKID-based Encryption bit shall be set to 1 in the Memory Encryption Features Enable field.
 - Memory Encryption Algorithm Select field has a single algorithm selected that the target will utilize for data-at-rest security. (The selected algorithm shall be one that the target supports, as reported by Get Target Capabilities Response.)
 - CKID Base = Base that the target supports.
 - Number of CKIDs = Number of CKIDs that the target supports – 1, so that CKID Base + Number of CKIDs is out of range.
- Host software receives Set Target Configuration Response.
- Host software issues Lock Target Configuration to make the configuration immutable.
- Host software receives Lock Target Configuration Response.
- Host software issues Set Target CKID Specific Key to associate a key with a CKID:
 - CKID is assigned within a valid range that the target supports.
 - CKID Type = OSCKID.
 - Validity Flags, Bit[0] set.
 - Data Encryption Key to utilize for the CKID.
- Host software receives Set Target CKID Specific Key Response.
- Host-untrusted (VM) software generates a full cacheline memory write request to the test address with a non-TEE opcode, CKID assigned, and known data pattern A.
- Host software verifies that the write returned a non-TEE opcode in the response.
- Host-untrusted (VM) software generates a memory read request to the same test address but with a non-TEE opcode and the assigned CKID.
- Host software verifies that the read returned a non-TEE opcode in the response and the data matches the known data pattern A:

// Attempt to write with CKID out of range
- Host-untrusted (VM) software generates a full cacheline memory write request to the same test address but with a non-TEE opcode and the CKID = CKID Base + Number of CKIDs, programmed in step 1, using a known data pattern B:

// Verify write was dropped
- Host-untrusted (VM) software generates a memory read request to the same test address but with a non-TEE opcode using the assigned CKID.
- Host software verifies that the read returned a non-TEE opcode in the response and the data matches the known data pattern A:

// Attempt to read with CKID out of range
- Host-untrusted (VM) software generates a memory read request to the same test address but with a non-TEE opcode with the CKID = CKID Base + Number of CKIDs programmed in step 1.

15. Host software verifies that the read returned a non-TEE opcode in the response and the data returns an all 1s pattern.

Pass Criteria:

- Read data returned for all reads is as expected

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Set Target CKID Specific Key results in a TSP Error Response
- Pass criteria is not met

14.11.7.10 Target-based CKID-based Memory Encryption Invalid CKID Type

This test verifies basic invalid CKID Type handling for optional target-based CKID-based memory encryption. The following TSP-related table is tested:

- [Table 11-27, “Target Behavior for Verifying CKID Type”](#)

Prerequisites:

- Device must support CXL.cachemem TSP security
- Device must support Compliance Mode DOE and SPDM over DOE
- Host software has established a secure SPDM link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for target-based CKID-based encryption

Topologies:

- SHDA

Test Steps:

1. Host software issues Set Target Configuration to enable target-based CKID-based memory encryption:
 - a. Encryption bit and CKID-based Encryption bit shall be set to 1 in the Memory Encryption Features Enable field.
 - b. Memory Encryption Algorithm Select field has a single algorithm selected that the target will utilize for data-at-rest security. (The selected algorithm shall be one that the target supports, as reported by Get Target Capabilities Response.)
 - c. For targets that limit the CKID range, CKID Base Required is set (see [Test 14.11.7.3](#)) and CKID Base and Number of CKIDs is set to a range that the target supports.
2. Host software receives Set Target Configuration Response.
3. Host software issues Lock Target Configuration to make the configuration immutable.
4. Host software receives Lock Target Configuration Response.
5. Host software issues Set Target CKID Specific Key to associate a key with a CKID:
 - a. CKID is assigned within a valid range that the target supports.
 - b. CKID Type = OSCKID.
 - c. Validity Flags, Bit[0] set.

- d. Data Encryption Key to utilize for the CKID.
6. Host software receives Set Target CKID Specific Key Response.
7. Host-untrusted (VM) software generates a full cacheline memory write request to the test address with the CKID assigned with a non-TEE opcode using a known data pattern A.
8. Host software verifies that the write returned a non-TEE opcode in the response.
9. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
10. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.
11. Host-trusted TEE (TVM) software generates a full cacheline memory write request to the same test address but with the CKID assigned with a TEE opcode using a known data pattern B.
12. Host software verifies that the write returned a non-TEE opcode in the response.
13. Host-trusted TEE (TVM) software generates a memory read request to the same test address but with the CKID assigned with a TEE opcode.
14. Host software verifies that the read returned a non-TEE opcode in the response and a fixed all 1s data pattern. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
15. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.
16. Host software issues Set Target CKID Specific Key to associate a key with a CKID:
 - a. CKID is assigned within valid range that the target supports.
 - b. CKID Type = TVMCKID.
 - c. Validity Flags, Bit[0] set.
 - d. Data Encryption Key to utilize for the CKID.
17. Host software receives Set Target CKID Specific Key Response.
18. Host-trusted TEE (TVM) software generates a full cacheline memory write request to the same test address but with the CKID assigned with a TEE opcode using a known data pattern A.
19. Host software verifies that the write returned a TEE opcode in the response.
20. Host-trusted TEE (TVM) software generates a memory read request to the same test address but with the CKID assigned with a TEE opcode.
21. Host software verifies that the read returned a TEE opcode in the response and expected data pattern A.
22. Host-untrusted (VM) software generates a full cacheline memory write request to the same test address but with the CKID assigned with a non-TEE opcode using a known data pattern B.
23. Host software verifies that the write returned a TEE opcode in the response.
24. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
25. Host software verifies that the read returned a TEE opcode in the response and a fixed all 1s data pattern.
26. Host-trusted TEE (TVM) software generates a memory read request to the same test address but with the CKID assigned with a TEE opcode.

27. Host software verifies that the read returned a TEE opcode in the response and expected data pattern A.

Pass Criteria:

- TEE/non-TEE opcode returned is correct for all writes and reads
- Read data returned for all reads is as expected

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Set Target CKID Specific Key results in a TSP Error Response
- Pass criteria is not met

14.11.7.11 Target-based CKID-based Memory Encryption Clearing Keys

This test verifies basic clear key handling for optional target-based CKID-based memory encryption.

Prerequisites:

- Device must support CXL.cachemem TSP security
- Device must support Compliance Mode DOE and SPDMA over DOE
- Host software has established a secure SPDMA link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for target-based CKID-based encryption

Topologies:

- SHDA

Test Steps:

1. Host software issues Set Target Configuration to enable target-based CKID-based memory encryption:
 - a. Encryption bit and CKID-based Encryption bit shall be set to 1 in the Memory Encryption Features Enable field.
 - b. Memory Encryption Algorithm Select field has a single algorithm selected that the target will utilize for data-at-rest security. (The selected algorithm shall be one that the target supports, as reported by Get Target Capabilities Response.)
 - c. For targets that limit the CKID range, CKID Base Required is set (see [Test 14.11.7.3](#)), and CKID Base and Number of CKIDs are both set to a range the target supports.
2. Host software receives Set Target Configuration Response.
3. Host software issues Lock Target Configuration to make the configuration immutable.
4. Host software receives Lock Target Configuration Response.
5. Host software issues Set Target CKID Random Key to associate a key with a CKID:
 - a. CKID is assigned within a valid range that the target supports.
 - b. CKID Type = OSCKID.
 - c. Validity Flags, Bit[0] set.

- d. Data Encryption Key Entropy X to utilize for the CKID.
6. Host software receives Set Target CKID Random Key Response.
7. Host-untrusted (VM) software generates a full cacheline memory write request to the test address with the CKID assigned with a non-TEE opcode using a known data pattern A.
8. Host software verifies that the write returned a non-TEE opcode in the response.
9. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
10. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.
11. Host software issues Clear Target CKID Key to disassociate a key from a CKID:
 - a. CKID assigned in valid range supported by the target.
12. Host software receives Clear Target CKID Key Response.
13. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
14. Host software verifies that the read returned a non-TEE opcode in the response and the expected data pattern is NOT data pattern A.
15. Host software issues Set Target CKID Random Key to associate a key with a CKID:
 - a. CKID is assigned within a valid range that the target supports.
 - b. CKID Type = OSCKID.
 - c. Validity Flags, Bit[0] set.
 - d. Data Encryption Key Entropy X to utilize for the CKID.
16. Host software receives Set Target CKID Random Key Response.
17. Host-untrusted (VM) software generates a memory read request to the same test address but with the CKID assigned with a non-TEE opcode.
18. Host software verifies that the read returned a non-TEE opcode in the response and the expected data pattern is NOT data pattern A.

Pass Criteria:

- TEE/non-TEE opcode returned is correct for all writes and reads
- Read data returned for all reads is as expected

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Set Target CKID Specific Key results in a TSP Error Response
- Clear Target CKID Key results in a TSP Error Response
- Pass criteria is not met

14.11.7.12 Target-based Range-based Memory Encryption

This test verifies the basic setting and clearing of encryption keys for optional target-based range-based memory encryption.

Prerequisites:

- Device must support CXL.cachemem TSP security

- Device must support Compliance Mode DOE and SPDM over DOE
- Host software has established a secure SPDM link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for target-based range-based encryption

Topologies:

- SHDA

Test Steps:

1. Host software issues Set Target Configuration to enable target-based range-based memory encryption:
 - a. Encryption bit and Range-based Encryption bit shall be set to 1 in the Memory Encryption Features Enable field.
 - b. Memory Encryption Algorithm Select field has a single algorithm selected that the target will utilize for data-at-rest security. (The selected algorithm shall be one that the target supports, as reported by Get Target Capabilities Response.)
2. Host software receives Set Target Configuration Response.
3. Host software issues Lock Target Configuration to make the configuration immutable.
4. Host software receives Lock Target Configuration Response.
5. Host software issues Set Target Range Specific Key to associate a key with the first memory range:
 - a. Range ID = 0.
 - b. Range Start and Range End fields describe the test address range on the host.
 - c. Validity Flags, Bit[0] set.
 - d. Data Encryption Key to utilize for the memory Range ID.
6. Host software receives Set Target Range Specific Key Response.
7. Host software generates a full cacheline memory write request for the test address range using a known data pattern A.
8. Host software generates a memory read request to the same test address range.
9. Host software verifies that the data matches the known data pattern A.
10. Host software issues Clear Target Range Key to remove the association of a key with the memory range:
 - a. Range ID = 0.
11. Host software receives Clear Target Range Key Response.
12. Host software generates a full cacheline memory write request for the test address range using a known data pattern B.
13. Host software generates a memory read request to the same test address range.
14. Host software verifies that the data does NOT match the known data pattern A or B.

Pass Criteria:

- Data does not match expected pattern after any reads

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Set Target Range Specific Key results in a TSP Error Response
- Clear Target Range Key results in a TSP Error Response
- Pass criteria is not met

14.11.7.13 Target-based Range-based Memory Encryption Clearing Keys

This test verifies basic clear key handling for optional target-based range-based memory encryption.

Prerequisites:

- Device must support CXL.cachemem TSP security
- Device must support Compliance Mode DOE and SPDM over DOE
- Host software has established a secure SPDM link to the device
- [Test 14.11.7.3](#) passed AND the target reports support for target-based range-based encryption

Topologies:

- SHDA

Test Steps:

1. Host software issues Set Target Configuration to enable target-based range-based memory encryption:
 - a. Encryption bit and Range-based Encryption bit shall be set to 1 in the Memory Encryption Features Enable field.
 - b. Memory Encryption Algorithm Select field has a single algorithm selected that the target will utilize for data-at-rest security. (The selected algorithm shall be one that the target supports, as reported by Get Target Capabilities Response.)
2. Host software receives Set Target Configuration Response.
3. Host software issues Lock Target Configuration to make the configuration immutable.
4. Host software receives Lock Target Configuration Response.
5. Host software issues Set Target Range Random Key to associate a key with a memory range:
 - a. Range ID = 0.
 - b. Range Start/Range End = Valid 4-KB-aligned block within the HDM memory range that contains the test address.
6. Host software receives Set Target Range Random Key Response.
7. Host-untrusted (VM) software generates a full cacheline memory write request to the test address with an address within the range for which the key was set with a non-TEE opcode using a known data pattern A.
8. Host software verifies that the write returned a non-TEE opcode in the response.
9. Host-untrusted (VM) software generates a memory read request to the same test address but with a non-TEE opcode.

10. Host software verifies that the read returned a non-TEE opcode in the response and expected data pattern A.
11. Host software issues Clear Target Range Key to disassociate the test address from the key:
 - a. Range ID = 0.
12. Host software receives Clear Target Range Key Response.
13. Host-untrusted (VM) software generates a memory read request to the same test address but with a non-TEE opcode.
14. Host software verifies that the read returned a non-TEE opcode in the response and the expected data pattern is NOT data pattern A.
15. Host software issues Set Target Range Random Key to associate a key with a memory range:
 - a. Range ID = 0.
 - b. Range Start/Range End = Valid 4-KB-aligned block within the HDM memory range that contains the test address.
16. Host software receives Set Target Range Random Key Response.
17. Host-untrusted (VM) software generates a memory read request to the same test address range but with a non-TEE opcode.
18. Host software verifies that the read returned a non-TEE opcode in the response and the expected data pattern is NOT data pattern A.

Pass Criteria:

- TEE/non-TEE opcode returned is correct for all writes and reads
- Read data returned for all reads is as expected

Fail Conditions:

- Set Target Configuration results in a TSP Error Response
- Lock Target Configuration results in a TSP Error Response
- Set Target Range Random Key results in a TSP Error Response
- Clear Target Range Key results in a TSP Error Response
- Pass criteria is not met

14.12**Reliability, Availability, and Serviceability**

RAS testing is dependent on being able to inject and correctly detect the injected errors. For this testing, it is required that the host and the device both support error injection capabilities.

Certain Device/Host capabilities of error injection are required to enable the RAS tests. First, the required capabilities and configurations are provided. Then, the actual test procedures are laid out. Because these capabilities may only be firmware accessible, currently these are implementation specific. However, future revisions of this specification may define these under an additional capability structure.

The following register describes the required functionalities. All the registers that have an RWL attribute should be locked when DVSEC Test Lock is set to 1.

Table 14-28. Register 1 — CXL.cacheMem LinkLayerErrorInjection

Bit	Attribute	Description
0	RWL	CachePoisonInjectionStart: Software writes 1 to this bit to trigger a single poison injection on a CXL.cache message in the Tx direction. Hardware must override the poison field in the data header slot of the corresponding message (D2H if device, H2D if Host). This bit is required only if CXL.cache protocol is supported.
1	RO	CachePoisonInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished poisoning a packet. Software is permitted to poll on this bit to determine when hardware has finished poison injection. This bit is required only if CXL.cache protocol is supported.
2	RWL	MemPoisonInjectionStart: Software writes 1 to this bit to trigger a single poison injection on a CXL.mem message in the Tx direction. Hardware must override the poison field in the data header slot of the corresponding message. This bit is required only if CXL.mem protocol is supported.
3	RO	MemPoisonInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished poisoning a packet. Software is permitted to poll on this bit to determine when hardware has finished poison injection. This bit is required only if CXL.mem protocol is supported.
4	RWL	IOPoisonInjectionStart: Software writes 1 to this bit to trigger a single poison injection on a CXL.io message in the Tx direction. Hardware must override the poison field in the data header slot of the corresponding message.
5	RO	IOPoisonInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished poisoning a packet. Software is permitted to poll on this bit to determine when hardware has finished poison injection.
7:6	RWL	CacheMemCRCInjection: Software writes to these bits to trigger CRC error injections. The number of CRC bits flipped is as follows: 00b = Disable. No CRC errors are injected. 01b = Single bit flipped in the CRC field for n subsequent Tx flits, where n is the value in CacheMemCRCInjectionCount. 10b = Two bits flipped in the CRC field for n subsequent Tx flits, where n is the value in CacheMemCRCInjectionCount. 11b = Three bits flipped in the CRC field for n subsequent Tx flits, where n is the value in CacheMemCRCInjectionCount. The specific bit positions that are flipped are implementation specific. This field is required if the CXL.cache protocol or CXL.mem protocol is supported.
9:8	RWL	CacheMemCRCInjectionCount: Software writes to these bits to program the number of CRC injections. This field must be programmed by software before OR at the same time as the CacheMemCRCInjection field. The number of flits where CRC bits are flipped is as follows: 00b = Disable. No CRC errors are injected. 01b = CRC injection is only for one flit. The CacheMemCRCInjectionBusy bit is cleared after one injection. 10b = CRC injection is for two flits in succession. The CacheMemCRCInjectionBusy bit is cleared after two injections. 11b = CRC injection is for three flits in succession. The CacheMemCRCInjectionBusy bit is cleared after three injections. This field is required if the CXL.cache protocol or CXL.mem protocol is supported.
10	RO	CacheMemCRCInjectionBusy: Hardware loads 1 to this bit when the associated Start bit is written. Hardware must clear this bit to indicate that it has indeed finished CRC injections. Software is permitted to poll on this bit to determine when hardware has finished CRC injection. This bit is required if the CXL.cache protocol or CXL.mem protocol is supported.

Table 14-29. Register 2 — CXL.io LinkLayer Error Injection

Bit	Attribute	Description
0	RWL	IOPoisonInjectionStart: Software writes 1 to this bit to trigger a single poison injection on a CXL.io message in the Tx direction. Hardware must override the poison field in the data header slot of the corresponding message.
1	RO	IOPoisonInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished poisoning a packet. Software is permitted to poll on this bit to determine when hardware has finished poison injection.
2	RWL	FlowControlErrorInjection: Software writes 1 to this bit to trigger a Flow Control error on CXL.io only. Hardware must override the Flow Control DLLP.
3	RO	FlowControlInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished Flow Control error injections. Software is permitted to poll on this bit to determine when hardware has finished Flow Control error injection.

Table 14-30. Register 3 — Flex Bus LogPHY Error Injections

Bit	Attribute	Description
0	RWL	CorrectableProtocolIDErrorInjection: Software writes 1 to this bit to trigger a correctable Protocol ID error on any CXL flit that is issued by the FlexBus LogPHY. Hardware must override the Protocol ID field in the flit.
1	RWL	UncorrectableProtocolIDErrorInjection: Software writes 1 to this bit to trigger an uncorrectable Protocol ID error on any CXL flit that is issued by the FlexBus LogPHY. Hardware must override the Protocol ID field in the flit.
2	RWL	UnexpectedProtocolIDErrorInjection: Software writes 1 to this bit to trigger an unexpected Protocol ID error on any CXL flit that is issued by the FlexBus LogPHY. Hardware must override the Protocol ID field in the flit.
3	RO	ProtocolIDInjectionBusy: Hardware loads 1 to this bit when the Start bit is written. Hardware must clear this bit to indicate that it has indeed finished Protocol ID error injections. Software is permitted to poll on this bit to determine when hardware has finished Protocol ID error injection. Software should only program one of the bits between the correctable, uncorrectable, and unexpected Protocol ID error injection bits.

14.12.1 RAS Configuration

14.12.1.1 AER Support

Prerequisites:

- Errors must be reported via the PCIe AER mechanism
- AER is as an optional Extended Capability

Test Steps:

1. Read through each Extended Capability (EC) Structure for the Endpoint, and then locate the EC structure for that type.

Pass Criteria:

- AER Extended Capability Structure exists

Fail Conditions:

- AER Extended Capability Structure does not exist

14.12.1.2 CXL.io Poison Injection from Device to Host

Prerequisites:

- CXL device must support Algorithm 1a
- CXL device must support Link Layer Error Injection capabilities for CXL.io

Test Steps:

1. Set up the device for Multiple Write streaming:
 - a. Write a pattern {64{8'hFF}} to cache-aligned Address A1.
 - b. Write a Compliance mode DOE to inject poison:

Table 14-31. CXL.io Poison Injection from Device to Host — I/O Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	6, Poison Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	0

- c. Write Compliance mode DOE with the following request:

Table 14-32. CXL.io Poison Injection from Device to Host — Multi-Write Streaming Request

Data Object Byte Offset	Length in Bytes	Description	Value
00h	8	Standard DOE Request Header	
08h	1	Request Code	3, Multiple Write Streaming
09h	1	Version	2
0Ah	2	Reserved	
0Ch	1	Protocol	0
0Dh	1	Virtual Address	0
0Eh	1	Self-checking	0
0Fh	1	Verify Read Semantics	0
10h	1	Num Increments	0
11h	1	Num Sets	0
12h	1	Num Loops	1
13h	1	Reserved	
14h	8	Start Address	A1
1Ch	8	Write Address	0
24h	8	WriteBackAddress	A2 (Must be distinct from A1)
2Ch	8	Byte Mask	FFFF FFFF FFFF FFFFh
34h	4	Address Increment	0
38h	4	Set Offset	0
3Ch	4	Pattern "P"	AAh
40h	4	Increment Pattern "B"	0

Pass Criteria:

- Receiver logs the poisoned received error
- Test software is permitted to read Address *A1* to observe the written pattern

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.3 CXL.cache Poison Injection**14.12.1.3.1 Device to Host Poison Injection****Prerequisites:**

- Device is CXL.cache capable
- CXL device must support Algorithm 1a
- CXL device must support Link Layer Error Injection capabilities for CXL.cache

Test Steps:

1. Set up the device for Multiple Write streaming:
 - a. Write a pattern {64{8'hFF}} to cache-aligned Address *A1*.
 - b. Write a Compliance mode DOE to inject poison:

Table 14-33. Device to Host Poison Injection — Cache Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	6, Poison Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

- c. Write Compliance mode DOE with the following request:

Table 14-34. Device to Host Poison Injection — Multi-Write Streaming Request (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description	Value
00h	8	Standard DOE Request Header	
08h	1	Request Code	3, Multiple Write Streaming
09h	1	Version	2
0Ah	2	Reserved	
0Ch	1	Protocol	1
0Dh	1	Virtual Address	0
0Eh	1	Self-checking	0
0Fh	1	Verify Read Semantics	0
10h	1	Num Increments	0
11h	1	Num Sets	0

Table 14-34. Device to Host Poison Injection — Multi-Write Streaming Request (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description	Value
12h	1	Num Loops	1
13h	1	Reserved	
14h	8	Start Address	A1
1Ch	8	Write Address	0
24h	8	WriteBackAddress	A2 (Must be distinct from A1)
2Ch	8	Byte Mask	FFFF FFFF FFFF FFFFh
34h	4	Address Increment	0
38h	4	Set Offset	0
3Ch	4	Pattern "P"	AAh
40h	4	Increment Pattern "B"	0

Pass Criteria:

- Receiver (host) logs the poisoned received error
- Test software is permitted to read Address A1 to observe the written pattern

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.3.2 Host to Device Poison Injection

This test ensures that if a CXL.cache device receives poisoned data from the host, the device returns the poison indication in the write-back phase. The Receiver on the CXL device must also log and escalate the poisoned received error.

Prerequisites:

- Device is CXL.cache capable
- CXL device must support Algorithm 1a with DirtyEvict and RdOwn semantics

Test Steps:

1. Repeatedly write a predetermined pattern to cacheline-aligned Address A1 (example pattern — all 1s — {64{8'hFF}}). A1 should belong to Host-attached memory.
 - a. Write a Compliance mode DOE to the host to inject poison:

Table 14-35. Host to Device Poison Injection — Cache Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	6, Poison Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

Pass Criteria:

- Receiver (device) logs the poisoned received error
- Test software is permitted to read Address A1 to observe the written pattern

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.4 CXL.cache CRC Injection**14.12.1.4.1 Device to Host CRC Injection****Test Equipment:**

- Protocol Analyzer

Prerequisites:

- Device is CXL.cache capable
- CXL device must support Algorithm 1a
- CXL device must support Link Layer Error Injection capabilities for CXL.cache

Test Steps:

1. Setup is the same as Test 14.3.6.1.2.
 - a. Write a Compliance mode DOE to inject CRC:

Table 14-36. Device to Host CRC Injection — Cache Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	7, CRC Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

Pass Criteria:

- Same as Test 14.3.6.1.2
- Monitor and verify that CRC errors are injected (using the Protocol Analyzer), and that Retries are triggered as a result

Fail Conditions:

- Same as Test 14.3.6.1.2

14.12.1.4.2 Host to Device CRC Injection**Test Equipment:**

- Protocol Analyzer

Prerequisites:

- Device is CXL.cache capable

- CXL device must support Algorithm 1a

Test Steps:

1. Setup is the same as Test 14.3.6.1.2.
 - a. While a test is running, software will periodically write a Compliance mode DOE to inject CRC:

Table 14-37. Host to Device CRC Injection — Cache Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	7, CRC Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

Pass Criteria:

- Same as Test 14.3.6.1.2
- Monitor and verify that CRC errors are injected (using the Protocol Analyzer), and that Retries are triggered as a result

Fail Conditions:

- Same as Test 14.3.6.1.2

14.12.1.5 CXL.mem Link Poison Injection

14.12.1.5.1 Host to Device Poison Injection

Prerequisites:

- Device is CXL.mem capable

Test Steps:

1. Write {64{8'hFF}} to Address *B1* from the host. *B1* must belong to device-attached memory.
2. Set up the **host** Link Layer for poison injection:
 - a. Write a Compliance mode DOE to the host to inject poison:

Table 14-38. Host to Device Poison Injection — Mem Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	6, Poison Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	3

3. Write {64{8'hAA}} to Address *B1* from the host.

Pass Criteria:

- Receiver (device) logs the poisoned received error
- Test software is permitted to read Address *B1* to observe the written pattern

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.6 CXL.mem CRC Injection**14.12.1.6.1 Host to Device CRC Injection****Test Equipment:**

- Protocol Analyzer

Prerequisites:

- Device is CXL.mem capable

Test Steps:

1. Write {64{8'hFF}} to Address *B1* from the host (*B1* must belong to device-attached memory).
2. Set up the **host** Link Layer for CRC injection.
 - a. Write a Compliance mode DOE to the host to inject CRC errors:

Table 14-39. Host to Device CRC Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	7, CRC Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	3

3. Write {64{8'hAA}} to Address *B1* from the host.
4. Read Address *B1* from the host, and compare to {64{8'hAA}}.

Pass Criteria:

- Read data == {64{8'hAA}}
- CRC error and Retry observed on the link (Protocol Analyzer used for observation)

Fail Conditions:

- Read data != {64{8'hAA}}

14.12.1.7 Flow Control Injection

This is an optional but strongly recommended test that is applicable only for CXL.io.

14.12.1.7.1 Device to Host Flow Control Injection

Prerequisites:

- CXL device must support Algorithm 1a
- CXL device must support Link Layer Error Injection capabilities

Test Steps:

1. Setup is the same as Test 14.3.6.1.1.
2. Write a Compliance mode DOE to the Device to set up flow control injection:

Table 14-40. Compliance Mode Flow Control Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	8, Flow Control Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

3. Write a Compliance DOE request to set up Algorithm 1a traffic:

Table 14-41. Compliance Mode Multiple Write Streaming Request

Data Object Byte Offset	Length in Bytes	Description	Value
00h	8	Standard DOE Request Header	
08h	1	Request Code	3, Multiple Write Streaming
09h	1	Version	2
0Ah	2	Reserved	
0Ch	1	Protocol	1
0Dh	1	Virtual Address	0
0Eh	1	Self-checking	0
0Fh	1	Verify Read Semantics	0
10h	1	Num Increments	0
11h	1	Num Sets	0
12h	1	Num Loops	1
13h	1	Reserved	
14h	8	Start Address	A1
1Ch	8	Write Address	0
24h	8	WriteBackAddress	A2 (Must be distinct from A1)
2Ch	8	Byte Mask	FFFF FFFF FFFF FFFFh
34h	4	Address Increment	0
38h	4	Set Offset	0
3Ch	4	Pattern "P"	AAh
40h	4	Increment Pattern "B"	0

Pass Criteria:

- Same as Test 14.3.6.1.1

Fail Conditions:

- Same as Test 14.3.6.1.1

14.12.1.7.2 Host to Device Flow Control Injection**Prerequisites:**

- CXL device must support Algorithm 1a

Test Steps:

1. Setup is the same as Test 14.3.6.1.1.
2. Write a Compliance mode DOE to the host to set up flow control injection:

Table 14-42. Compliance Mode Flow Control Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	8, Flow Control Injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

3. Write {64{8'hAA}} to Address B1 from the host.
4. Read Address B1 from the host, and compare to {64{8'hAA}}.

Pass Criteria:

- Same as Test 14.3.6.1.1

Fail Conditions:

- Same as Test 14.3.6.1.1

14.12.1.8 Unexpected Completion Injection

This is an optional but strongly recommended test that is applicable only for CXL.io.

14.12.1.8.1 Device to Host Unexpected Completion Injection**Prerequisites:**

- CXL device must support Algorithm 1a
- CXL device must support Device Error Injection capabilities

Test Steps:

1. Write a Compliance mode DOE to set up Unexpected Completion:

Table 14-43. Compliance Mode Unexpected MAC Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	Bh, Unexpected MAC injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

2. Write a Compliance DOE request to set up Algorithm 1a traffic:

Table 14-44. Compliance Mode Multiple Write Streaming Request

Data Object Byte Offset	Length in Bytes	Description	Value
00h	8	Standard DOE Request Header	
08h	1	Request Code	3, Multiple Write Streaming
09h	1	Version	2
0Ah	2	Reserved	
0Ch	1	Protocol	1
0Dh	1	Virtual Address	0
0Eh	1	Self-checking	0
0Fh	1	Verify Read Semantics	0
10h	1	Num Increments	0
11h	1	Num Sets	0
12h	1	Num Loops	1
13h	1	Reserved	
14h	8	Start Address	A1
1Ch	8	Write Address	0
24h	8	WriteBackAddress	A2 (Must be distinct from A1)
2Ch	8	Byte Mask	FFFF FFFF FFFF FFFFh
34h	4	Address Increment	0
38h	4	Set Offset	0
3Ch	4	Pattern "P"	AAh
40h	4	Increment Pattern "B"	0

Pass Criteria:

- Unexpected completion error is logged

Fail Conditions:

- No errors are logged

14.12.1.9 Completion Timeout

This is an optional but strongly recommended test that is applicable only for CXL.io.

14.12.1.9.1 Device to Host Completion Timeout

Prerequisites:

- CXL device must support Algorithm 1a
- CXL device must support Device Error Injection capabilities

Test Steps:

1. Write a Compliance mode DOE to the Device to set up MAC Delay injection:

Table 14-45. Compliance Mode MAC Delay Injection Request

Data Object Byte Offset	Length in Bytes	Description	Value
0h	8	Standard DOE Request Header	
8h	1	Request Code	Ah, MAC Delay injection
9h	1	Version	2
Ah	2	Reserved	
Ch	1	Protocol	1

2. Write a Compliance DOE request to set up Algorithm 1a traffic:

Table 14-46. Compliance Mode Multiple Write Streaming Request (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description	Value
00h	8	Standard DOE Request Header	
08h	1	Request Code	3, Multiple Write Streaming
09h	1	Version	2
0Ah	2	Reserved	
0Ch	1	Protocol	1
0Dh	1	Virtual Address	0
0Eh	1	Self-checking	0
0Fh	1	Verify Read Semantics	0
10h	1	Num Increments	0
11h	1	Num Sets	0
12h	1	Num Loops	1
13h	1	Reserved	
14h	8	Start Address	A1
1Ch	8	Write Address	0
24h	8	WriteBackAddress	A2 (Must be distinct from A1)
2Ch	8	Byte Mask	FFFF FFFF FFFF FFFFh
34h	4	Address Increment	0

Table 14-46. Compliance Mode Multiple Write Streaming Request (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description	Value
38h	4	Set Offset	0
3Ch	4	Pattern "P"	AAh
40h	4	Increment Pattern "B"	0

Pass Criteria:

- Completion timeout is logged and escalated to the error manager

Fail Conditions:

- No errors are logged and data corruption is seen

14.12.1.10 CXL.mem Media Poison Injection**14.12.1.10.1 Host to Memory Device Poison Injection****Prerequisites:**

- Device is CXL.mem capable

Test Steps:

- Select error injection target address Device Physical Address (DPA) that belongs to the DUT.
- Translate the DPA to the Host Physical Address (HPA).
- Request Poison error injection via Enable Memory Device Poison Injection DOE specifying the DPA where the error is to be injected.
- Poll on the Poison Injection Response DOE. Successful completion status indicates that the device has injected the poison into memory.
- Host performs a memory read at the error injection target HPA and the device responds to the read with the poison indicator set.

Pass Criteria:

- Receiver (device) logs the poisoned received error
- When injecting poison into persistent memory regions of the CXL.mem device:
 - The device shall add the new physical address to the device's poison list and the error source should be set to an injected error and reported through the **Get Poison List** command
 - In addition, the device should add an appropriate poison creation event to its internal Informational Event Log, update the Event Status register, and if configured, interrupt the host
 - Poison shall be persistent across warm reset or cold reset until explicitly cleared by overwriting the cacheline with new data with the poison indicator cleared

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.11 CXL.mem LSA Poison Injection

14.12.1.11.1 Host to Memory Device LSA Poison Injection

Prerequisites:

- Device is CXL.mem capable

Test Steps:

1. Select error injection LSA byte offset that belongs to the DUT.
2. Request LSA Poison error injection via Enable Memory Device LSA Poison Injection Compliance DOE, specifying the LSA byte offset where the error is to be injected.
3. Poll on the Poison Injection Response DOE. Successful completion status indicates that the device has injected the poison into memory.
4. Host performs a GetLSA mailbox command that includes the LSA byte offset at which the poison was injected into the LSA. The device responds to the read with an error in the mailbox GetLSA command and appropriate error log generation.

Pass Criteria:

- Receiver (device) errors the GetLSA command to the injected LSA byte offset
- When injecting poison into the persistent memory Label Storage Area of the CXL.mem device:
 - Device should add an appropriate poison creation event to its internal Informational Event Log, update the Event Status register, and if configured, interrupt the host
 - Poison shall be persistent across warm reset or cold reset until explicitly cleared by a SetLSA with new data that overwrites the poisoned data at the original poison injection LSA byte offset

Fail Conditions:

- Receiver does not log the poisoned received error

14.12.1.12 CXL.mem Device Health Injection

14.12.1.12.1 Host to Device Poison Injection

Prerequisites:

- Applicable only for devices that support Device Health Injection with the DOE transport
- Device is CXL.mem capable

Test Steps:

1. Request device health injection via Enable CXL.mem Device Health Injection Compliance DOE, specifying the health status field to inject.
2. Poll on the Poison Injection Response DOE. Successful completion status indicates that the device has injected the health status change into the device.
3. Host verifies device health status changes by inspecting Event Log Records and device health status changes.

Pass Criteria:

- Device notifies host of state change through appropriate Event Log Records, and the resulting change in device health can be verified through the Get Health Info command

Fail Conditions:

- Receiver does not see correct event logs or change in health status

14.13 Memory Mapped Registers**14.13.1 CXL Capability Header****Test Steps:**

1. The base address for these registers is at Offset 4K from the Register Base Low and Register Base High found in the Register Locator DVSEC.
2. Read Offset 00h, Length 4 bytes.
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
23:20	CXL_Cache_Mem_Version
31:24	Array_Size

4. Save the Array_Size to be used for finding the remaining capability headers in the subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0001h	Always
CXL_Capability_Version	1h	Always
CXL_Cache_Mem_Version	1h	Always

Pass Criteria:

- Test 14.8.2 passed
- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.2 CXL RAS Capability Header**Test Steps:**

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

31:20 CXL_RAS_Capability_Pointer

4. Save CXL_RAS_Capability_Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0002h	Always
CXL_Capability_Version	2h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.3 CXL Security Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL_Security_Capability_Pointer
4. Save CXL_Security_Capability_Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0003h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.4 CXL Link Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL_Link_Capability_Pointer

4. Save CXL_Link_Capability_Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0004h	Always
CXL_Capability_Version	2h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.5 CXL HDM Decoder Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.

2. Read this element (1 DWORD).

3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL_HDM_Decoder_Capability_Pointer

4. Save CXL_HDM_Decoder_Capability_Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0005h	Always
CXL_Capability_Version	3h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.6 CXL Extended Security Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.

2. Read this element (1 DWORD).

3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL_Extended_Security_Capability_Pointer

4. Save CXL_Extended_Security_Capability_Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0006h	Always
CXL_Capability_Version	2h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.7 CXL IDE Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL IDE Capability Pointer

4. Save CXL IDE Capability Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0007h	Always
CXL_Capability_Version	2h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.8 CXL HDM Decoder Capability Register

Prerequisites:

- HDM Decoder Capability is implemented

Test Steps:

1. Read register, CXL_HDM_Interleave_Capability_Pointer + Offset 00h, Length 2 bytes.
2. Decode this into:

Bits	Variable
3:0	Decoder Count
7:4	Target Count

3. Verify:

Variable	Value	Condition
Decoder Count	< Dh	Always
Target Count	< 9h	Always

Pass Criteria:

- 14.13.5 Device Present passed
- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.9 CXL HDM Decoder Commit**Prerequisites:**

- HDM Decoder Capability is implemented

Test Steps:

1. Program an address range in the Decoder[m+1].Base and Decoder[m+1].Size registers such that:
 - Decoder[m+1].Base $\geq$ (Decoder[m].Base+Decoder[m].Size), and
 - Decoder[m+1].Base $\leq$ Decoder[m+1].Base+Decoder[m+1].Size
2. Program distinct Target Port Identifiers for Interleave Way=0 through $2^{**}IW - 1$ (not applicable to Devices).
3. Set the Commit bit in the Decoder[m+1].Control register.

Pass Criteria:

- Committed bit in the Decoder[m+1].Control register is set
- Error Not Committed bit in the Decoder[m+1].Control register is not set

Fail Conditions:

- Committed bit in the Decoder[m+1].Control register is not set within 10 ms
- Error Not Committed bit in the Decoder[m+1].Control register is set

14.13.10 CXL HDM Decoder Zero Size Commit**Prerequisites:**

- HDM Decoder Capability is implemented

Test Steps:

1. Program 0 in the Decoder[m].Size register.
2. Set the Commit bit in the Decoder[m].Control register.

Pass Criteria:

- Committed bit in the Decoder[m].Control register is set
- Error Not Committed bit in the Decoder[m].Control register is not set

Fail Conditions:

- Committed bit in the Decoder[m].Control register is not set within 10 ms
- Error Not Committed bit in the Decoder[m].Control register is set

14.13.11 CXL Snoop Filter Capability Header**Test Steps:**

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL Snoop Filter Capability Pointer
4. Save CXL Snoop Filter Capability Pointer, which is used in subsequent tests.
5. Verify

Variable	Value	Condition
CXL_Capability_ID	0008h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.12 CXL Device Capabilities Array Register

This test locates all the CXL Device Capability Headers, in addition to the Verify Conditions listed below.

Test Steps:

1. The base address for this register is obtained from the Register Locator DVSEC.
2. Read Offset 00h, Length 8 bytes.
3. Decode this into:

Bits	Variable
15:0	Capability ID
19:16	Version
47:32	Capabilities Count

4. Verify:

Variable	Value	Condition
Capability ID	0000h	Always
Version	01h	Always

5. For N, where N ranges from 1 through Capabilities_Count, find each Capability Header Element by reading 2 bytes at Offset N*10h.
6. Decode this into:

Bits Variable

15:0 CXL_Capability_ID_Arr[N]

7. If CXL_Capability_ID_Arr[N] == 0001h, save Offset N*10h as Device_Status_Registers_Capabilities_Header_Base.
8. If CXL_Capability_ID_Arr[N] == 0002h, save Offset N*10h as Primary_Mailbox_Registers_Capabilities_Header_Base.
9. If CXL_Capability_ID_Arr[N] == 0003h, save Offset N*10h as Secondary_Mailbox_Registers_Capabilities_Header_Base.
10. If CXL_Capability_ID_Arr[N] == 4000h, save Offset N*10h as Memory_Device_Status_Registers_Capabilities_Header_Base.

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.13 Device Status Registers Capabilities Header Register**Test Steps:**

1. Read offset Device_Status_Registers_Capabilities_Header_Base, Length 4 bytes. Device_Status_Registers_Capabilities_Header_Base is obtained in the test performed in Test 14.13.12.
2. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

3. Verify:

Variable	Value	Condition
CXL_Capability_ID	0001h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.14 Primary Mailbox Registers Capabilities Header Register**Test Steps:**

1. Read offset Primary_Mailbox_Registers_Capabilities_Header_Base, Length 4 bytes. Primary_Mailbox_Registers_Capabilities_Header_Base is obtained in the test performed in Test 14.13.12.
2. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

3. Verify:

Variable	Value	Condition
CXL_Capability_ID	0002h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.15 Secondary Mailbox Registers Capabilities Header Register

Test Steps:

1. Read offset Secondary_Mailbox_Registers_Capabilities_Header_Base, Length 4 bytes. Secondary_Mailbox_Registers_Capabilities_Header_Base is obtained in the test performed in Test 14.13.12.
2. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

3. Verify:

Variable	Value	Condition
CXL_Capability_ID	0003h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.16 Memory Device Status Registers Capabilities Header Register

Test Steps:

1. Read offset Memory_Device_Status_Registers_Capabilities_Header_Base, Length 4 bytes. Memory_Device_Status_Registers_Capabilities_Header_Base is obtained in the test performed in Test 14.13.12.
2. Find the CXL Device Capability Header register as described in Test 14.13.12, step 5, Length 4 bytes.
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

4. Verify:

Variable	Value	Condition
CXL_Capability_ID	4000h	Always

CXL_Capability_Version

1h

Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.17 CXL Timeout and Isolation Capability Header**Prerequisites:**

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL_Timeout_and_Isolation_Capability_Pointer

4. Save CXL_Timeout_and_Isolation_Capability_Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	09h	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.18 CXL.cachemem Extended Register Header**Prerequisites:**

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version

31:20 CXL.cachemem Extended Register Capability Pointer

4. Save CXL.cachemem Extended Register Capability Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Ah	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.19 CXL BI Route Table Capability Header

Prerequisites:

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL BI Route Table Capability Pointer

4. Save CXL BI Route Table Capability Pointer, which is used in subsequent tests.

5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Bh	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.20 CXL BI Decoder Capability Header

Prerequisites:

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.

2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL BI Decoder Capability Pointer
4. Save CXL BI Decoder Capability Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Ch	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.21 CXL Cache ID Route Table Header**Prerequisites:**

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL Cache ID Route Table Capability Pointer
4. Save CXL Cache ID Route Table Capability Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Dh	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.22 CXL Cache ID Decoder Capability Header

Prerequisites:

- Device supports 256B Flit mode and 256B Flit mode is enabled

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL Cache ID Local Decoder Capability Pointer

4. Save CXL Cache ID Local Decoder Capability Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Eh	Always
CXL_Capability_Version	01h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.23 CXL Extended HDM Decoder Capability Header

Test Steps:

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL Extended HDM Decoder Capability Pointer

4. Save CXL Extended HDM Decoder Capability Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0Fh	Always
CXL_Capability_Version	03h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.13.24 CXL Extended Metadata Capability Header**Test Steps:**

1. Find this capability by reading all the elements within the Array_Size.
2. Read this element (1 DWORD).
3. Decode this into:

Bits	Variable
15:0	CXL_Capability_ID
19:16	CXL_Capability_Version
31:20	CXL Extended Metadata Capability Pointer
4. Save CXL Extended Metadata Capability Pointer, which is used in subsequent tests.
5. Verify:

Variable	Value	Condition
CXL_Capability_ID	0010h	Always
CXL_Capability_Version	1h	Always

Pass Criteria:

- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.14 Memory Device Tests

This section covers tests that are applicable to devices that support the CXL.mem protocol.

14.14.1 DVSEC CXL Range 1 Size Low Registers**Prerequisites:**

- Not applicable to FM-owned LD
- Device is CXL.mem capable

Test Steps:

1. Read Configuration Space for DUT, CXL_DEVICE_DVSEC_BASE + Offset 1Ch, Length 2 bytes.
2. Decode this into:

Bits	Variable
7:5	Memory_Class
12:8	Desired_Interleave
3. Verify:

Variable	Value	Condition
Memory_Class	010b	Always

Desired_Interleave	00h, 01h, or 02h	Always
Desired_Interleave	03h, 04h, 05h, 06h, or 07h	CXL 2.0 and higher

Pass Criteria:

- Test 14.8.4 passed
- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.14.2 DVSEC CXL Range 2 Size Low Registers**Prerequisites:**

- Not applicable to FM-owned LD
- Device is CXL.mem capable
- HDM_Count=10b

Inputs:

- **Type:** Volatile or Non-volatile
- **Class:** Memory or Storage

Test Steps:

1. Read Configuration Space for DUT, CXL_DEVICE_DVSEC_BASE + Offset 2Ch, Length 2 bytes.
2. Decode this into:

Bits	Variable
4:2	Media_Type
7:5	Memory_Class
12:8	Desired_Interleave

3. Verify:

Variable	Value	Condition
Media_Type	000b, 001b, or 010b	Always
Memory_Class	000b, 001b, or 010b	Always
Desired_Interleave	0h, 1h or 2h	Always
Desired_Interleave	03h, 04h, 05h, 06h, or 07h	CXL 2.0 and higher

Pass Criteria:

- Test 14.8.4 passed
- Verify Conditions are met

Fail Conditions:

- Verify Conditions failed

14.15 Sticky Register Tests

This section covers tests applicable to registers that are sticky through a reset.

14.15.1 Sticky Register Test

Test Steps:

1. Read and record the value of the following ROS registers for step 5:

Error Capabilities and Control Register (Offset 14h)

Bits	Variable
5:0	First_Error_Pointer

Header Log Registers (Offset 18h)

Bits	Variable
511:0	Header Log

Note:

Register contents may or may not be 0.

2. Set the following RWS registers to settings as per list and record the written values for step 5.

RWS Registers and settings:

Uncorrectable Error Mask Register (Offset 04h)

Bits	Variable	Settings
11:0	Error Mask registers	Set to FFFh
14:14	Internal_Error_Mask	Set to 1
15:15	CXL_IDE_Tx_Mask	Set to 1
16:16	CXL_IDE_Rx_Mask	Set to 1

Uncorrectable Error Severity Register (Offset 08h)

Bits	Variable	Settings
11:0	Error Severity registers	Set to FFFh
14:14	Internal_Error_Severity	Set to 1
15:15	CXL_IDE_Tx_Severity	Set to 1
16:16	CXL_IDE_Rx_Severity	Set to 1

Correctable Error Mask Register (Offset 10h)

Bits	Variable	Settings
6:0	Error Mask registers	Clear to 00h

Error Capabilities and Control Register (Offset 14h)

Bits	Variable	Settings
13:13	Poison_Enabled	Set to 1

CXL Link Layer Capability Register (Offset 00h)

Bits	Variable	Settings
3:0	CXL Link Version Supported	Set to 2h
15:8	LLR Wrap Value Supported	Set to FFh

Note:

Intention is to set the register to a nonzero value.

CXL Link Layer Control and Status Register (Offset 08h)

Bits	Variable	Settings
1:1	LL_Init_Stall	Set to 1
2:2	LL_Crd_Stall	Set to 1

CXL Link Layer Rx Credit Control Register (Offset 10h)

Bits	Variable	Settings
9:0	Cache Req Credits	Set to 3FFh
19:10	Cache Rsp Credits	Set to 3FFh
29:20	Cache Data Credits	Set to 3FFh
39:30	Mem Req_Rsp Credits	Set to 3FFh
49:40	Mem Data Credits	Set to 3FFh
59:50	BI Credits	Set to 3FFh

CXL Link Layer Ack Timer Control Register (Offset 28h)

Bits	Variable	Settings
7:0	Ack Force Threshold	Set to FFh
17:8	Ack or CRD Flush Retimer	Set to 1FFh

CXL Link Layer Defeature Register (Offset 30h)

Bits	Variable	Settings
0:0	MDH Disable	Set to 1

DVSEC CXL Control2 (Offset 10h)

Bits	Variable	Settings
4:4	Desired Volatile HDM State after Hot Reset	Set to 1 if DVSEC CXL Capability3 (Offset 38h) bit[3] Volatile HDM State after Hot Reset - Configurability == 1

3. Issue a link Hot Reset.
4. Wait for the link to train back to CXL.
5. Verify:
 - a. ROS register values before and after link Hot Reset are matching.
 - b. RWS register values before and after link Hot Reset are matching.

Pass Criteria:

- Test 14.8.2 passed
- Verify Conditions are met

Fail Conditions:

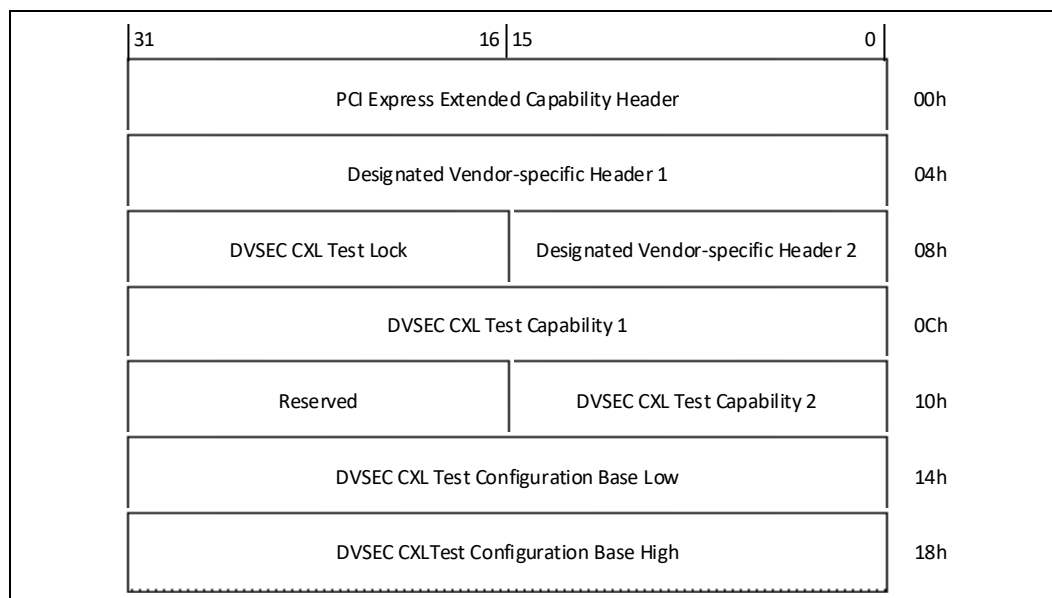
- Verify Conditions failed

14.16 Device Capability and Test Configuration Control

Implementations are expected to provision a Data Object Exchange (DOE) Interface to enable compliance capabilities.

14.16.1 CXL Device Test Capability Advertisement

Figure 14-19. PCIe DVSEC for Test Capability



To advertise Test capabilities, the standard DVSEC register fields should be set as below:

Table 14-47. DVSEC Registers

Register	Bit Location	Field	Value
Designated Vendor-Specific Header 1 (Offset 04h)	15:0	DVSEC Vendor ID	1E98h
	19:16	DVSEC Revision	0h
	31:20	DVSEC Length	1Ch
Designated Vendor-Specific Header 2 (Offset 08h)	15:0	DVSEC ID	00Ah

Table 14-48. DVSEC CXL Test Lock (Offset 0Ah)

Bit	Attribute	Description
0	RWO	TestLock: Software writes 1 to lock the relevant test configuration registers.
15:1	N/A	Reserved

Table 14-49. DVSEC CXL Test Configuration Base Low (Offset 14h)

Bit	Attribute	Description
0	RO	MemorySpaceIndicator: The test configuration registers are in memory space. Device must hardwire this bit to 0.
2:1	RO	Type 00b = Base register is 32 bits wide and can be mapped anywhere in the 32-bit address space 01b = Reserved 10b = Base register is 64 bits wide and can be mapped anywhere in the 64-bit address space 11b = Reserved
3	RO	Reserved: Device must hardwire this bit to 0.
31:4	RW	BaseLow: Bits [31:4] of the base address where the test configuration registers exist.

Table 14-50. DVSEC CXL Test Configuration Base High (Offset 18h)

Bit	Attribute	Description
31:0	RW	BaseHigh: Bits[63:32] of the base address where the test configuration registers exist.

14.16.2 Debug Capabilities in Device

14.16.2.1 Error Logging

The following capabilities in a device are strongly recommended to support ease of verification and compliance testing.

A device that supports self-checking must include an error status and header log register with the following fields:

Table 14-51. Register 9 – ErrorLog1 (Offset 40h)

Bit	Attribute	Description
31:0	RW	ExpectedPattern: Expected data pattern as per device hardware.
63:32	RW	ObservedPattern: Observed data pattern as per device hardware.

Table 14-52. Register 10 – ErrorLog2 (Offset 48h)

Bit	Attribute	Description
31:0	RW	ExpectedPattern: Expected data pattern as per device hardware.
63:32	RW	ObservedPattern: Observed data pattern as per device hardware.

Table 14-53. Register 11 — ErrorLog3 (Offset 50h)

Bit	Attribute	Description
7:0	RW	ByteOffset: First byte offset within the cacheline where the data mismatch was observed.
15:8	RW	LoopNum: Loop number where data mismatch was observed.
16	RW1C	ErrorStatus: Set to 1 by device if data mismatch was observed.

14.16.2.2 Event Monitors

It is strongly recommended that a device advertise at least two event monitors, which can be used to count device-defined events. An event monitor consists of two 64-bit registers:

- An event controller: EventCtrl
- An event counter: EventCount

The usage model is for software to program EventCtrl to count an event of interest, and then read the EventCount to determine how many times the event has occurred. At a minimum, a device must implement the ClockTicks event. When the ClockTicks event is selected via the event controller, the event counter will increment once every clock cycle, based on the application layer's clock. Further suggested events may be published in the future. Examples of other events that a device may choose to implement are:

- Number of times a particular opcode is sent or received
- Number of retries or CRC errors
- Number of credit returns sent or received
- Device-specific events that may help visibility on the platform or with statistical calculation of performance

Table 14-54 and Table 14-55 list the EventCtrl and EventCount register formats, respectively.

Table 14-54. Register 12 — EventCtrl (Offset 60h)

Bit	Attribute	Description
7:0	RW	EventSelect: Field that is used to select which of the available events should be counted in the paired EventCount register ¹ .
15:8	RW	SubEventSelect: Field that is used to select which sub-conditions of an event should be counted in the paired EventCount register ¹ . This field is a bitmask, where each bit represents a different condition. The EventCount should increment if any of the selected sub-conditions occurs. For example, an event might be Transactions Received, with three sub-conditions of Read, Write and Completion.
16	N/A	Reserved
17	RW	Reset: When set to 1, the paired EventCount register ¹ will be cleared to all 0s. Writing a 0 to this bit has no effect.
18	RW	EdgeDetect • 0 = Counter will increment once within each cycle that the event has occurred • 1 = Counter will increment when a 0 to 1 transition (i.e., rising edge) is detected
63:19	N/A	Reserved

1. See Table 14-55.

Table 14-55. Register 13 — EventCount (Offset 68h)

Bit	Attribute	Description
63:0	RO	EventCount: Hardware load register that is updated with a running count of the occurrences of the event programmed in the EventCtrl register. The count monotonically increases unless software explicitly writes the count to a lower value or writes to the Reset bit in the paired EventCtrl register (see Table 14-54).

14.16.3 Compliance Mode DOE

Function 0 of a CXL device must support the DOE mailbox for the compliance modes to be controlled through it. The Vendor ID must be set to the CXL Vendor ID to indicate that this Object type is defined by the CXL specification. The Data Object Type must be cleared to 00h to advertise that this is a Compliance Mode type of data object.

Table 14-56. Compliance Mode — Data Object Header

Bits Location	Field	Value
15:0	Vendor ID	1E98h
23:16	Data Object Type	00h

Table 14-57. Compliance Mode Return Values

Value	Description	Value	Description
0000 0000h	Success	0000 0005h	Target Busy
0000 0001h	Not Authorized	0000 0006h	Target Not Initialized
0000 0002h	Unknown Failure	0000 0007h	Invalid Address Specified
0000 0003h	Unsupported Injection Function	0000 0008h	Invalid Injection Parameter
0000 0004h	Internal Error		

14.16.3.1 Compliance Mode Capability

Request and response pair for determining the device's compliance capabilities.

Table 14-58. Compliance Mode Availability Request

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0, Query Capabilities.
9h	1	Version of Capability Requested: Supply 0 here for the highest supported Compliance DOE version, or specify a specific version (e.g., 3).
Ah	2	Reserved

Table 14-59. Compliance Mode Availability Response

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Response Code: Value is 0, Query Capabilities.
09h	1	Version of Capability Returned: Returns supported version of the Compliance mode DOE, by the spec revision number (e.g., 3).
0Ah	1	Length of Capability Package
0Bh	1	Status: See Table 14-57 for error codes.
0Ch	8	Available Compliance Capabilities Bitmask
14h	8	Enabled Compliance Capabilities Bitmask
1Ch	8	Compliance Capabilities Options: See Table 14-60 for Compliance Option value descriptions.

The Available Capabilities and Enabled Capabilities bitmask values correspond to the request codes of each capability. For example:

- Bit[1] will be set if the DOE supports Request Code 1 (Status)
- Bit[3] will be set if the DOE supports Request Code 3 (Multiple Write Streaming)

Table 14-60. Compliance Options Value Descriptions

Bits	Description
15:0	Write Semantics Supported: Bitmask with the corresponding values: • Bit[0]: Set to 1 if Device supports CXL.cache and ItoMWr opcodes as requester • Bit[1]: Set to 1 if Device supports CXL.cache and WrCur opcodes as requester • Bit[2]: Set to 1 if Device supports CXL.cache and DirtyEvict opcodes as requester • Bit[3]: Set to 1 if Device supports CXL.cache and WOWrInv opcodes as requester • Bit[4]: Set to 1 if Device supports CXL.cache and WOWrInvF opcodes as requester • Bit[5]: Set to 1 if Device supports CXL.cache and WrInv opcodes as requester • Bit[6]: Set to 1 if Device supports CXL.cache and CLFlush opcodes as requester • Bit[7]: Set to 1 if Device supports CXL.cache and CleanEvict opcodes as requester • Bit[8]: Set to 1 if Device supports CXL.cache and CleanEvictNoData opcodes as requester • Bits[15:9]: Reserved
31:16	Read Semantics Supported: Bitmask with the corresponding values: • Bit[16]: Set to 1 if Device supports CXL.cache and RdCurr opcodes as requester • Bit[17]: Set to 1 if Device supports CXL.cache and RdOwn opcodes as requester • Bit[18]: Set to 1 if Device supports CXL.cache and RdShared opcodes as requester • Bit[19]: Set to 1 if Device supports CXL.cache and RdAny opcodes as requester • Bit[20]: Set to 1 if Device supports CXL.cache and RdOwnNoData opcodes as requester • Bits[31:21]: Reserved
47:32	• Bit[32]: Set to 1 if Device supports CXL.cache and CacheFlushed opcodes as requester • Bits[47:33]: Reserved
63:48	Reserved

The Available Compliance Capabilities and Enabled Compliance Capabilities bitmask values correspond to the request codes of each capability. For example:

- Bit[1] will be set if the DOE supports Request Code 1 (Status)
- Bit[3] will be set if the DOE supports Request Code 3 (Multiple Write Streaming)

14.16.3.2 Compliance Mode Status

Shows which compliance mode capabilities are enabled or in use.

Table 14-61. Compliance Mode Status

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 1, Query Status.
9h	1	Version of Capability Requested
Ah	2	Reserved

Table 14-62. Compliance Mode Status Response

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Header
08h	1	Response Code: Value is 1, Query Status.
09h	1	Version of Capability Returned
0Ah	1	Length of Capability Package
0Bh	4	Capability Bitfield
0Eh	2	Cache Size
10h	1	Cache Size Units

14.16.3.3 Compliance Mode Halt All

Table 14-63. Compliance Mode Halt All

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 2, Halt All.
9h	1	Version of Capability Requested
Ah	2	Reserved

Table 14-64. Compliance Mode Halt All Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 2, Halt All.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.4 Compliance Mode Multiple Write Streaming

Table 14-65. Enable Multiple Write Streaming Algorithm on the Device

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: Value is 3, Multiple Write Streaming.
09h	1	Version of Capability Requested
0Ah	2	Reserved
0Ch	1	Protocol
0Dh	1	Virtual Address
0Eh	1	Self-checking
0Fh	1	Verify Read Semantics
10h	1	Num Increments
11h	1	Num Sets
12h	1	Num Loops
13h	1	Reserved
14h	8	Start Address
1Ch	8	Write Address
24h	8	Writeback Address
2Ch	8	Byte Mask
34h	4	Address Increment
38h	4	Set Offset
3Ch	4	Pattern "P"
40h	4	Increment Pattern "B"

Table 14-66. Compliance Mode Multiple Write Streaming Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 3, Multiple Write Streaming.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

If the device only supports Virtual Addresses and the Virtual Address is cleared to 0, then the return value shall be 01h (Not Authorized). If the device only supports Physical Addresses and the Virtual Address is set to 1, then the return value shall be 01h (Not Authorized).

14.16.3.5 Compliance Mode Producer-Consumer

Table 14-67. Enable Producer-Consumer Algorithm on the Device

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: Value is 4, Producer-Consumer.
09h	1	Version of Capability Requested
0Ah	2	Reserved
0Ch	1	Protocol
0Dh	1	Num Increments
0Eh	1	Num Sets
0Fh	1	Num Loops
10h	1	Write Semantics
11h	3	Reserved
14h	8	Start Address
1Ch	8	Byte Mask
24h	4	Address Increment
28h	4	Set Offset
2Ch	4	Pattern

Table 14-68. Compliance Mode Producer-Consumer Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 4, Producer-Consumer.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

If the device only supports Virtual Addresses and the Virtual Address is cleared to 0, then the return value shall be 01h (Not Authorized). If the device only supports Physical Addresses and the Virtual Address is set to 1, then the return value shall be 01h (Not Authorized).

14.16.3.6 Test Algorithm 1b Multiple Write Streaming with Bogus Writes

Table 14-69. Enable Algorithm 1b, Write Streaming with Bogus Writes (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: Value is 5, Test Algorithm 1b.
09h	1	Version of Capability Requested

Table 14-69. Enable Algorithm 1b, Write Streaming with Bogus Writes (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description
0Ah	2	Reserved
0Ch	1	Protocol
0Dh	1	Virtual Address
0Eh	1	Self-checking
0Fh	1	Verify Read Semantics
10h	1	Num Increments
11h	1	Num Sets
12h	1	Num Loops
13h	1	Reserved
14h	8	Start Address
1Ch	8	Writeback Address
24h	8	Byte Mask
2Ch	4	Address Increment
30h	4	Set Offset
34h	4	Pattern "P"
38h	4	Increment Pattern "B"
3Ch	1	Bogus Writes Count
3Dh	3	Reserved
40h	4	Bogus Writes Pattern

Table 14-70. Algorithm 1b Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 5, Test Algorithm 1b.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.7 Inject Link Poison

Table 14-71. Enable Poison Injection into (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 6, Poison Injection.
9h	1	Version of Capability Requested

Table 14-71. Enable Poison Injection into (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description
Ah	2	Reserved
Ch	1	Protocol 00h = CXL.io 01h = CXL.cache 02h = CXL.mem

Table 14-72. Poison Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 6, Poison Injection.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.8 Inject CRC

Table 14-73. Enable CRC Error into Traffic

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 7, CRC Injection.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	Num Bits Flipped
Dh	1	Num Flits Injected

Table 14-74. CRC Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 7, CRC Injection.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.9 Inject Flow Control

Table 14-75. Enable Flow Control Injection

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 8, Flow Control Injection.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	Inject Flow Control

Table 14-76. Flow Control Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 8, Flow Control Injection.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.10 Toggle Cache Flush

Table 14-77. Enable Cache Flush Injection

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 9, Cache Flush.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	00h = Cache Flush Disabled 01h = Cache Flush Enabled

Table 14-78. Cache Flush Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 9, Cache Flush.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

14.16.3.11 Inject MAC Delay

Delay MAC on IDE secure link until it no longer meets spec, flit 6+.

Table 14-79. MAC Delay Injection

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0Ah, Delay MAC.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	00h = Disable 01h = Enable MAC Delay
Dh	1	Mode 00h = CXL.io 01h = CXL.cachemem
Eh	1	Delay: Number of flits to delay MAC. 6+ = error condition.

Table 14-80. MAC Delay Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Ah, MAC delay injection.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status 00h = Success - See Table 14-57 for other error codes

14.16.3.12 Insert Unexpected MAC

Insert an unexpected MAC on a non-IDE secured channel.

Table 14-81. Unexpected MAC Injection

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0Bh, Unexpected MAC injection.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	00h = Disable 01h = Insert message 02h = Delete message
Dh	1	Mode 00h = CXL.io 01h = CXL.cachemem

Table 14-82. Unexpected MAC Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Bh, Unexpected MAC injection.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package

14.16.3.13 Inject Viral

Table 14-83. Enable Viral Injection

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0Ch, Inject Viral.
9h	1	Version
Ah	2	Reserved
Ch	1	Protocol 00h = CXL.io 01h = CXL.cache 02h = CXL.mem

Table 14-84. Flow Control Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Ch, Inject Viral.
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status 00h = Success - See Table 14-57 for other error codes

14.16.3.14 Inject ALMP in Any State

Insert an ALMP in the ARB/MUX regardless of state.

Table 14-85. Inject ALMP Request (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0Dh, Inject ALMP in any state.
9h	1	Version of Capability Requested

Table 14-85. Inject ALMP Request (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description
Ah	2	Reserved
Ch	1	00h = Disable 01h = Insert ALMP
Dh	3	Reserved

Table 14-86. Inject ALMP Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Dh, Inject ALMP in any state.
9h	1	Version of Capability Returned
Ah	6	Reserved

14.16.3.15 Ignore Received ALMP

Ignore the next ALMPs received.

Table 14-87. Ignore Received ALMP Request

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code = 0Eh, Ignore received ALMPs.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	00h = Disable 01h = Ignore ALMPs
Dh	3	Reserved

Table 14-88. Ignore Received ALMP Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Eh, Ignore received ALMPs.
9h	1	Version of Capability Returned
Ah	6	Reserved

14.16.3.16 Inject Bit Error in Flit

Inject a single bit error into the lower 16 bytes of a 528-bit flit.

Table 14-89. Inject Bit Error in Flit Request

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Request Code: Value is 0Fh, Inject Bit Error in Flit.
9h	1	Version of Capability Requested
Ah	2	Reserved
Ch	1	00h = Disable; no action taken 01h = Inject single Bit error in next 528 flits
Dh	3	Reserved

Table 14-90. Inject Bit Error in Flit Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 0Fh, Inject Bit Error in Flit.
9h	1	Version of Capability Returned
Ah	6	Reserved

14.16.3.17 Inject Memory Device Poison

Table 14-91. Memory Device Media Poison Injection Request (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: Value is 10h, Memory Device Media Poison Injection.
09h	1	Version of Capability Requested
0Ah	2	Reserved
0Ch	1	Protocol 02h = CXL.mem
0Dh	1	Reserved
0Eh	1	Action 00h = Inject Poison 01h = Clear Poison
0Fh	1	Reserved

Table 14-91. Memory Device Media Poison Injection Request (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description
10h	8	Device Physical Address: When Protocol = 2, the device shall inject poison into the media at this requested address. If this address specifies a persistent memory address, the injected poison shall persist across warm resets or cold resets. Device shall report Invalid Address Specified poison injection response status if the DPA is out of range. - Bits[5:0]: Reserved - Bits[7:6]: DPA[7:6] - Bits[15:8]: DPA[15:8] ... - Bits[63:56]: DPA[63:56]
18h	40h	Clear Poison Write Data: When Protocol = 2 and Action = 1, the device shall write this replacement data into the requested physical address, atomically, while clearing poison. If the device is configured with nonzero Metadata bits as defined by the HDM-H Metabits Storage Configuration field in the Metabits Storage Feature Readable Attributes (see Table 8-282), then for a subsequent read to the DPA, the device shall return MetaField=00b (Meta0-State (MS0)) and MetaValue=00b.

Table 14-92. Memory Device Media Poison Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 10h, Memory Device Media Poison Injection.
9h	1	Version of Capability Returned
Ah	6	Reserved

Table 14-93. Memory Device LSA Poison Injection Request

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: Value is 11h, Memory Device LSA Poison Injection.
09h	1	Version of Capability Requested
0Ah	2	Reserved
0Ch	1	Protocol 02h = CXL.mem
0Dh	1	Reserved
0Eh	1	Action 00h = Inject Poison 01h = Clear Poison
0Fh	1	Reserved
10h	4	LSA Byte Offset: When Protocol = 2, the device shall inject poison into the Label Storage Area of the device at this requested byte offset. Because the LSA is persistent, the injected poison shall persist across warm resets or cold resets. Device shall report Invalid Address Specified poison injection response status if the byte offset is out of range. The poison can be cleared through this interface or with SetLSA.

Table 14-94. Memory Device LSA Poison Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: Value is 11h, Memory Device LSA Poison Injection.
9h	1	Version of Capability Returned
Ah	6	Reserved

Table 14-95. Inject Memory Device Health Enable Memory Device Health Injection (Sheet 1 of 2)

Data Object Byte Offset	Length in Bytes	Description
00h	8	Standard DOE Request Header
08h	1	Request Code: 12h, Memory Device Health Injection
09h	1	Version of Capability Requested
0Ah	2	Reserved
0Ch	1	Protocol 02h = CXL.mem
0Dh	1	Injection Type 00h = Error is injected immediately and remains in effect until it is cleared using this command or by a CXL warm reset or cold reset of the device 01h = Error is not injected until after a cold reset, the injection will only occur once, and will be auto-disabled after the first occurrence
0Eh	1	Valid Device Health Injection: Indicators of which Device Health Injection fields are valid in the supplied in the payload. - Bit[0]: 1 = Health Status Injection Enabled field shall be valid - Bit[1]: 1 = Media Status Injection Enabled field shall be valid - Bit[2]: 1 = Life Used Injection Enabled field shall be valid - Bit[3]: 1 = Dirty Shutdown Count Injection Enabled field shall be valid - Bit[4]: 1 = Device Temperature Injection Enabled field shall be valid - Bits[7:5]: Reserved

Table 14-95. Inject Memory Device Health Enable Memory Device Health Injection (Sheet 2 of 2)

Data Object Byte Offset	Length in Bytes	Description
0Fh	1	Enable Device Health Injection: The device shall enable the following error injection: • Bit[0]: Health Status Injection Enabled: – 0 = Device shall disable its Health Status injection – 1 = Health Status field shall be valid and the device shall enable its Health Status injection • Bit[1]: Media Status Injection Enabled: – 0 = Device shall disable its Media Status injection – 1 = Media Status field shall be valid and the device shall enable its Media Status injection • Bit[2]: Life Used Injection Enabled: – 0 = Device shall disable its Life Used injection – 1 = Life Used field shall be valid and the device shall enable its Life Used injection • Bit[3]: Dirty Shutdown Count Injection Enabled: – 0 = Device shall disable its Dirty Shutdown Count injection – 1 = Dirty Shutdown Count field shall be valid and the device shall enable its Dirty Shutdown Count injection • Bit[4]: Device Temperature Injection Enabled: – 0 = Device shall disable its Device Temperature injection – 1 = Device Temperature field shall be valid and the device shall enable its Device Temperature injection • Bits[7:5]: Reserved
10h	1	Health Status: The injected Health Status. One of the defined Get Health Info values from Section 8.2.10.9 . Return Invalid Injection Parameter for invalid or unsupported injection values.
11h	1	Media Status: The injected Media Status. One of the defined Get Health Info values from Section 8.2.10.9 . Return Invalid Injection Parameter for invalid or unsupported injection values.
12h	1	Life Used: The injected Life Used. See the Get Health Info command in Section 8.2.10.9 for legal range. Return Invalid Injection Parameter for invalid or unsupported injection values.
13h	1	Reserved
14h	4	Dirty Shutdown Count: The injected Dirty Shutdown Count. See the Get Health Info command in Section 8.2.10.9 . Return Invalid Injection Parameter for invalid or unsupported injection values.
18h	2	Device Temperature: The injected Device Temperature. See the Get Health Info command in Section 8.2.10.9 . Return Invalid Injection Parameter for invalid or unsupported injection values.

Table 14-96. Device Health Injection Response

Data Object Byte Offset	Length in Bytes	Description
0h	8	Standard DOE Request Header
8h	1	Response Code: 12h, Device Health Injection
9h	1	Version of Capability Returned
Ah	1	Length of Capability Package
Bh	1	Status: See Table 14-57 for error codes.

§ §

Appendix A Taxonomy

This appendix was included in the original release of CXL and has not been updated since. It is being included as reference for some original usage and implementation options/ideas for CXL devices. It does not cover features that were added after the initial release, including Back-Invalidate Snoop (BISnp) messages that enable new ways of handling coherence of Host-managed Device Memory (HDM), and new memory device expansion proposals around pooling and Fabric-Attached memory (FAM). See [Chapter 2.0, “CXL System Architecture,”](#) for a more-complete set of use cases.

A.1 Accelerator Usage Taxonomy

Table A-1. Accelerator Usage Taxonomy (Sheet 1 of 2)

Accelerator Type	Description	Challenges and Opportunities	CXL Support
Producer-Consumer Accelerators that don't execute against "Memory" without special requirements	- Works on data streams or large contiguous data objects - Little interaction with the host - Standard P/C ordering model works well	- Efficient work submission - Efficient exchange of metadata (flow control)	- Basic PCIe* - CXL.io
Producer-Consumer Plus Accelerators that don't execute against "Memory" with special requirements	- Same as above, but... - P/C ordering model doesn't work well - Needs special data operations such as atomics	Device Coherency can be used to implement varied ordering models and special data operations	- CXL.cache on CXL with baseline snoop filter support - CXL.io
Software-assisted SVM Memory Accelerators that execute against "Memory" with software-supportable data management	- Local memory is often needed for bandwidth or latency predictability - Little interaction with the host - Data management is easily implemented in software (e.g., few and simple data buffers)	- Host software should be able to interact directly with accelerator memory (e.g., SVM, Google, etc.) - Reduces copies, replication, and pinning - Optimizing coherency impact on performance is a challenge - Software can provide best optimization of coherency impact	- CXL Bias model with software-managed bias - CXL.io - CXL.cache - CXL.mem
Autonomous SVM Memory Accelerators that execute against "Memory" where software-supported data management is impractical	- Local memory is often needed for bandwidth or latency predictability - Interaction with the host is common - Data movement is difficult to manage in software (e.g., sparse data structures, pointer-based data structures, etc.)	- Host software should be able to interact directly with accelerator memory (SVM) - Reduces copies, replication, and pinning - Optimizing coherency impact on performance is a challenge - Cannot count on software for bias management	- CXL Bias model with hardware-managed bias - CXL.io - CXL.cache - CXL.mem

Table A-1. Accelerator Usage Taxonomy (Sheet 2 of 2)

Accelerator Type	Description	Challenges and Opportunities	CXL Support
Giant Cache Accelerators that execute against "Memory" where local memory and caching is required	- Local memory is needed for bandwidth or latency predictability - Data footprint is larger than local memory - Interaction with the host is common - Data must be cycled through accelerator memory in small blocks - Data movement is difficult to manage in software	- Accelerator memory needs to work like a cache (not SVM/system memory) - Ideally, cache misses are detected in hardware, but cache replacements can be managed in software	- CXL.cache on CXL with "Enhanced Directory" snoop filter support - CXL.io - CXL.cache
Disaggregated Memory Controller Typically for memory controllers with remote persistent memory, which may be in 2 Level-Memory or App Direct mode	- PCIe semantics are needed for device enumeration, driver support, and device management - Most operational flows rely on being able to communicate directly with a Home Device or Near Memory Controller on the Host	Device needs high-bandwidth and low-latency path from memory controller to Home Device in the CPU	- CXL.io - CXL.mem

A.2 Bias Model Flow Example — From CPU

- Start with pages in Device Bias:
 - Pages are guaranteed to not be cached in host cache hierarchy
- Software allocates pages from device memory:
 - Software pushes operands to allocated pages from the peer CPU core:
 - For example, Software may use OpenCL* API to flip operand pages to Host Bias
 - Data copies or cache flushes are not required
 - Host CPUs generate operand data in target pages — data arrives in an arbitrary location within the host cache hierarchy.
- Device uses operands to generate results:
 - For example, Software may use OpenCL API to flip operand pages back to Device Bias
 - API call causes a work descriptor submission to the device — descriptor asks the device to flush operand pages from the host cache.
 - Cache flush is executed using RdOwnNoData on CXL.cache protocol (see [Table 3-22](#)).
 - When Device Bias flip is complete, Software submits work to the accelerator.
 - Accelerator executes without any host-related coherency overhead.
 - Accelerator dumps data to results pages.
- Software pulls results from the allocated pages:
 - For example, Software uses OpenCL API to flip results pages to Host Bias
 - This action causes some bias states to be changed, but does not cause any coherency or cache-flushing actions
 - Host CPUs can access, cache, and share results data as needed
- Software releases the allocated pages.

OpenCL defines a Coarse-grained buffer Shared Virtual Memory model. Under that model, memory consistency is guaranteed only at explicit synchronization points and these points provide an opportunity to perform bias flip.

Here are some example of OpenCL calls where bias flip can be performed:

- `clEnqueueSVMMMap` provides host access to this buffer. Software may flip the bias from Device bias to Host bias during this call.
- `clEnqueueSVMUnmap` revokes host access to this buffer. At this point, an OpenCL implementation for a CXL device could flip the bias from Host bias to Device bias.

There are other OpenCL calls where the CPU and the Device share OpenCL buffer objects. Software could flip the bias during those calls.

A.3

CPU Support for Bias Modes

There are two envisaged models of support that the CPU would provide for Bias Modes. These are described below.

A.3.1

Remote Snoop Filter

- Remote socket-owned lines that belong to accelerator-attached memory are tracked by a Remote Snoop-Filter (SF) located in the host CPU Home Agent (HA). Remote SF does not track lines that belong to Host memory. The above removes the need for a directory in device memory. Please note this is only possible in Host Bias mode since in Device Bias mode, local/remote sockets can't cache lines that belong to device memory.
- Local socket-owned lines that belong to accelerator-attached memory will be tracked by a local SF in the host CPU Last Level Cache (LLC) controller. Please note this is only possible in Host Bias mode since in Device Bias mode, local/remote sockets can't cache lines that belong to device memory.
- Device-owned lines that belong to accelerator-attached memory (in Host Bias mode) will NOT be tracked by a local SF in the host CPU LLC controller. These will be tracked by the Device Coherency Engine (DCOH) using a device-specific mechanism (device SF). In Device Bias mode, no coherent tracking is done in the host CPU because the accesses are completed within the device and the host does not see the requests.
- Device-owned lines that belong to host memory (in Host mode or Device mode) will be tracked by a local SF in the host CPU LLC controller. This may cause the device to receive snoops through CXL (CXL.cache) for such lines.

A.3.2

Directory in Accelerator-attached Memory

- Remote socket-owned lines that belong to device memory are tracked by a directory in device memory metadata. The host HA may choose to do broadcast snooping for some cases to avoid reading the metadata.
- Local socket-owned lines that belong to device memory will be tracked by a local SF in the host CPU LLC controller. For access by device, local socket-owned lines that belong to device memory will also update the directory.
- Device-owned lines that belong to device memory will NOT be tracked by a local SF in the host CPU LLC controller. These will be tracked by the Device Coherency Engine (DCOH) using a device-specific mechanism (device SF).
- Device-owned lines that belong to host memory (in Host mode or Device mode) will be tracked by a local SF in the host CPU LLC controller. This may cause the device to receive snoops through CXL (CXL.cache) for such lines.

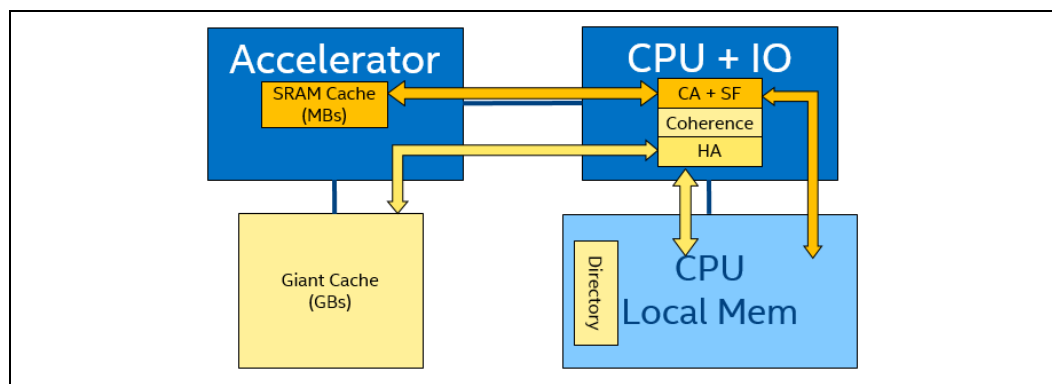
A.4

Giant Cache Model

For problems whose data sets exceed the device-attached memory size, the memory attached to the accelerator needs to be a cache, not memory:

- Typically, the full data set will reside in processor-attached memory
- Subsets of this larger data set are cycled through accelerator memory as the calculation proceeds
- For such use cases, caching is the correct solution:
 - Accelerator memory is not mapped into the system address map — data set is built up in host memory
 - Single-page table entry per page in data set — no page table manipulation as pages are cycled through accelerator memory
 - Copies of data can be created under driver control and/or hardware control with no OS intervention

Figure A-1. Profile D — Giant Cache Model



Critical issues with a Giant Cache:

- Cache is too large for tracking in the Host on-die SF
- Snoop latency for a Giant Cache is likely to be much higher than standard on-die cache snoop latency

Recommended CXL solution:

- Implements SF in processor's coherency directory (stored in DRAM ECC bits), which essentially becomes a highly scalable SF
- Minimizes impact to processor operations that are unrelated to accelerators
- Allows accelerator to access data over CXL.cache as a caching Device
- Provides support on CXL.cache to allow an accelerator to explicitly request directory snoop filtering for Giant Cache
- Processor infrastructure differentiates between low-latency and high-latency requester types
- Support for simultaneous use of a small, low-latency cache associated with the on-die snoop filter is included

Appendix B Unordered I/O to Support Peer-to-Peer Directly to HDM-DB

Revision 3.0 of the CXL specification added this appendix to document the planned CXL use of Unordered I/O (UIO) protocol for the CXL UIO Direct P2P to HDM-DB use case. At the time, UIO protocol was still under development within the PCI-SIG* as an ECN against the PCIe* Base Specification. The PCIe UIO ECN became final in March 2023, and is available from the PCI-SIG website.

The CXL use of UIO protocol as originally described in this appendix as well as additional use cases are covered elsewhere within this version of the CXL specification. UIO protocol details should be obtained from the final PCIe UIO ECN or other PCI-SIG specifications.

§ §

Appendix C Memory Protocol Tables

To formalize the protocol expectations of the Memory Protocol, this appendix captures the allowed request encodings, states, and responses in the host and the device. In this appendix, the flows are referred to in the context of three HDM region types as defined in [Section 3.3](#).

This appendix uses field name abbreviations to fit into the table format captured in [Table C-1](#).

Table C-1. Field Encoding Abbreviations (Sheet 1 of 2)

Field Name	Encoding Name	Abbreviation	Notes
MetaField	Meta0-State	MS0	
	Extended Meta-State	EMS	
	Meta0-State OR Extended Meta-State	MS0-EMS	
	Meta0-State OR No-Op	MS0-NO	
	Extended Meta-State OR No-Op	EMS-NO	For table rows that allow these two encodings in a DRS response, the No-Op encoding should be sent only when the device has an uncorrectable error on the Extended Metadata Stored such that it cannot return it.
DRS Trailer and RWD Trailer	Extended MetaValue	EMV	Extended MetaValue in the trailer. For DRS Trailer, this state comes from EMD state stored in the device at the time the request is received. For RWD Trailer, this is the state to become the final EMD state when the write is complete.
	Not Applicable	N/A	Indicates that no trailer is returned
	Extended MetaValue or Not Applicable	EMV-NA	Indicates that a trailer of EMV is sent, but only if the MetaField encoding is EMS. The table may allow for No-Op to be sent in MetaField, in which case no trailer is included.
MetaValue	Not Applicable	N/A	Used when MetaValue is undefined and should never be consumed, which is when MetaField is set to No-Op. The encoding rules follow the definition for Reserved which must be cleared to 0 by sender and ignored by receiver.
	Any Value	<Any>	Indicates any legal encoding that matches rules for the HDM memory region. In support of Extended Metadata with HDM-DB, the device must return the current host state (same as Meta0-State).
	Explicit No-Op	E-No-Op	This is an encoding defined for use only when EMV encoding to indicate the MetaField= No-Op equivalent coherent behavior in the device for HDM-DB.
Host State	UnChanged	UC	Used to indicate that the device should not change its bias state.

Table C-1. Field Encoding Abbreviations (Sheet 2 of 2)

Field Name	Encoding Name	Abbreviation	Notes
SnpType	Snp*	All Snoop types: SnpInv, SnpData, SnpCur	Used when a row applies to all different snoop options.
Device State	Extended Metadata	EMD	Data stored in the device for up to 32 bits of meta state. This mode is mutually exclusive with standard Meta0-State in HDM-H devices, but supplemental to coherence state in HDM-DB devices. Not applicable to HDM-D.

Note:

The term “Host State” is used in this section to describe the device tracking of the host coherence state for the address. The implementation of this may vary and has been referred to as a “Bias Table” or “Bias State” or “Directory” for HDM-D/HDM-DB. For HDM-DB more implementations are possible, including an inclusive Snoop-Filter, which uses the BISnp/BIRsp channels to keep the Snoop Filter inclusive.

In the tables that follow, “Y(1)” in the **Legal** column indicates that the table row contents are also defined in other tables within the CXL specification and also existed in the CXL 1.1 specification. A “Y*” in the **Legal** column indicates that a destination must accept the message, and the rows of the table indicate the legal responses to the message. The “O” defines optional rows that may not be supported. All “O” cases will include details as to the discovery/enabling method in [Table C-2](#).

Table C-2. Optional Codes

Option Code	Option Name	Description
O-1	Cmp-M Support	Mode that can be enabled in a device. This message is optional for the host to support and is thus expected to be applied to CXL.mem traffic from a given initiator (host or peer device). For MLD/GFD, this mode could be different for each initiator.
O-2	EMD Support	This mode would apply to a given HDM region. Hosts are expected to support EMD or Traditional Metadata for each region of HDM-H. For HDM-DB, the MetaValue is treated in the same way (defining coherence state); however, the EMD value is also supported.
O-3	MemSpecRd Support	These messages are supported only on interfaces that train as 256B or that are supported by links that train as 256B Flit mode or as CXL 68B Flit and VH Capable.
O-4	TEE Requests	TSP features are enabled in the host.

C.1

HDM-DB Requests with TEE Support

[Table C-3](#) defines messages on the request channel of CXL.mem protocol. [Table C-4](#) additionally defines the Forward flows that apply only for HDM-D memory regions for Type 2 devices when the devices are accessing device-attached memory. [Table C-5](#) defines the BISnp channel method of managing device-attached memory coherence for the HDM-DB memory region of Type 2 devices or Type 3 devices.

[Table C-3](#) includes a footnote that disallows the downgrade of “host state” from the current state if there is a TE mismatch detected at the target for read access. For example, if the current state is A and the normal resulting state is S-state, it is not permitted to downgrade to S-state if there is a TE State mismatch.

Use of the term UnChanged Mismatch (“UCM”) reflects cases in which the device cache or requester state in the HDM-DB target remains unchanged when a TEE State mismatch occurs. Targets that take advantage of this allowance will rely on software clean-up of any coherence violation that may result. For additional details related to requirements for this behavior, see [Section 11.5.4.11](#).

Table C-3. HDM-DB Memory Requests with TE State (Sheet 1 of 6)

Legal	Host Request				Device Response					Final Device State			Description	
	M2S Req	MetaField	MetaValue	SnpType	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD		
Y	MemRd <Only for Non-TSP targets>	MS0	I	SnpInv	Cmp	MemData	MS0-NO	<any>	N/A	I	I	UC	The host is requesting a non-cacheable but current value of the cacheline and is forcing the device to flush its cache.	
O-2		EMS					EMS-NO		EMV-NA					
N					SnpData									
Y		MS0		SnpCur	Cmp		MS0-NO	<any>	N/A	<any>	I	UC	The host is requesting a non-cacheable but current value of the cacheline and is thereby leaving data in the device's cache.	
O-2		EMS											EMS-NO	EMV-NA
N					No-Op									
O-1	MemRd/ MemRdTEE <Add description of target allowance for check or not for these opcodes as part of coherence resolution>	MS0	A	SnpInv	Cmp-M ²	MemData/ MemDataTEE	MS0-NO	<any>	N/A	I or UCM	A or UCM	UC	Supported for host that can accept M-state.	
O-1 & O-2		EMS					EMS-NO		EMV-NA					
Y(1)		MS0			Cmp-E		MS0-NO		N/A				The host wants an exclusive copy of the cacheline.	
O-2		EMS					EMS-NO		EMV-NA					
N					SnpData									
N					SnpCur									
N					No-Op									
N					SnpInv									
Y(1)		MS0	S	SnpData	Cmp-S		MS0-NO	<any>	N/A	S or UCM	S ³ or UCM	UC	The host is requesting a shared copy of the cacheline, but Rsp types allow the device to return S-state or E-state to the host. Cmp-E response is not recommended because the host did not request this state.	
O-2		EMS					EMS-NO		EMV-NA					
Y(1)		MS0			Cmp-E		MS0-NO		N/A					
O-2		EMS					EMS-NO		EMV-NA					
N					SnpCur									
N					No-Op									

Table C-3. HDM-DB Memory Requests with TE State (Sheet 2 of 6)

Legal	Host Request				Device Response					Final Device State			Description		
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD			
Y(1)	MemRd/ MemRdTEE	No-Op	N/A	SnptypeInv	Cmp	MemData/ MemDataTEE	MS0-NO	<any>	N/A	I or UCM	UC	UC	The host wants to read the cacheline without changing the state expected in the host cache, and the device should invalidate the cacheline from its cache.		
O-2		EMS	No-Op				EMS-NO		EMV-NA						
N		EMS-NO	N/A	SnptypeData											
N		EMS	E-No-Op												
Y(1)		No-Op	N/A	SnptypeCur	Cmp		MS0-NO	<any>	N/A	<any>	UC		The host wants a current value of the cacheline without changing the state expected in the host cache.		
O-2		EMS	E-No-Op				EMS-NO		EMV-NA						
Y			No-Op	N/A	No-Op		Cmp	MS0-NO	<any>	N/A	<any>	UC	UC	The host wants the value of the memory location without snooping the device cache and without changing the cache state expected in the host cache. A use case for this would be if the host includes E-state or S-state without data so that the host is requesting data only and does not want to change the cache state, and because the host has E-state or S-state, the host can know that the device cache does not need to be snooped.	
O-2		EMS	E-No-Op	EMS-NO				EMV-NA							
Y			<all>	<all>	<all>		<none>	MemData-NXM	No-Op	N/A	N/A	N/A	N/A	N/A	The special case MemData-NXM response is used if the fabric or the device is unable to positively decode the Address. This is a common response type for both Type 2 devices and Type 3 devices to avoid an ambiguous case in which the host is unsure of whether the host should expect an NDR message. See Section 3.3.11 for additional details.

Table C-3. HDM-DB Memory Requests with TE State (Sheet 3 of 6)

Legal	Host Request				Device Response					Final Device State			Description
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD	
Y(1)	MemInv/ MemInvTEE <Add description of target allowance for check or not for these opcodes>	MS0	A	Snptype	Cmp-E ⁴	<none>	<any>	<any>	N/A	I or UCM	A or UCM	UC	The host wants ownership of the cacheline but does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
N				Snptype									
Y			S	Snptype	Cmp-S ⁴		<any>	<any>		S or I or UCM	S or UCM	UC	The host wants the device to degrade to S-state in its caches, and wants the shared state for the cacheline (but does not require the data).
N				Snptype									
N				Snptype									
Y(1)			I	Snptype	Cmp		<any>	<any>		I or UCM	I	UC	The host wants the device to invalidate the cacheline from its caches and does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
Y		No-Op	N/A	Snptype	Cmp		<any>	<any>		I or UCM	UC	UC	The host wants the device to invalidate the cacheline from its caches and does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
N		EMS	N/A										

Table C-3. HDM-DB Memory Requests with TE State (Sheet 4 of 6)

Legal	Host Request				Device Response					Final Device State			Description
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD	
Y(1)	MemInvP/ MemInvPTEE <Target required to look up TE State>	MS0	A	Snptype	Cmp-E CmpTEE-E ⁴	<none>	<any>	<any>	N/A	I or UCM	A or UCM	UC	The host wants ownership of the cacheline but does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
N				Snptype									
Y			S	Snptype	Cmp-S/ CmpTEE-S ⁴		<any>	<any>		S or I or UCM	S or UCM	UC	The host wants the device to degrade to S-state in its caches, and wants the shared state for the cacheline (but does not require the data).
N				Snptype									
N				Snptype									
Y(1)			I	Snptype	Cmp/ CmpTEE		<any>	<any>		I or UCM	I	UC	The host wants the device to invalidate the cacheline from its caches and does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
Y		No-Op	N/A	Snptype	Cmp/ CmpTEE		<any>	<any>		I or UCM	UC	UC	The host wants the device to invalidate the cacheline from its caches and does not require the data.
N				Snptype									
N				Snptype									
N				Snptype									
N		EMS	N/A										

Table C-3. HDM-DB Memory Requests with TE State (Sheet 5 of 6)

Legal	Host Request				Device Response					Final Device State			Description
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD	
N	MemRdData/ MemRdDataTEE	<all>	N/A	SnptypeInv		MemData/ MemDataTEE							
Y(1)		MS0-NO		SnptypeData	Cmp-E		MS0	I or A	N/A	I or UCM	A or UCM	UC	The host wants a cacheable copy in either E-state or S-state.
O-2		EMS					EMS		EMV				
Y(1)		MS0-NO			Cmp-S		MS0	S	N/A	I or S or UCM	S ³ or UCM		
O-2		EMS					EMS		EMV				
Y					Cmp-E		No-Op	N/A	N/A	I or UCM	A or UCM		
Y					Cmp-S					I or S or UCM	S ³ or UCM		
N				SnptypeCur									
N				No-Op									
Y				<all>	<all>		<none>	MemData-NXM	No-Op	N/A	N/A	N/A	UC
N		MemSpecRd/ MemSpecRdTEE	MS0-EMD	<all>	<all>								
N					Snptype*								
O-3	No-Op		N/A	No-Op	<none>	<none>	<none>	<none>	N/A	UC	UC	UC	Speculative memory read. A Demand read following this with the same address will be merged in the device. No completion is expected for this transaction. Completion is returned with demand read.
N	MemClnEvtU <Requester does not know TE State>		A or S	<all>									
Y		MS0	I	No-Op	Cmp	<none>	No-Op	N/A	N/A	UC	I	UC	The host will only issue this request from E-state or from S-state. Target should make best estimate of TE State if required for coherence resolution.
N				Snptype*									
N		EMS-NO	N/A	<all>									

Table C-3. HDM-DB Memory Requests with TE State (Sheet 6 of 6)

Legal	Host Request				Device Response					Final Device State			Description
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Device Cache	DTRCS ¹	EMD	
N	MemClnEvct/ MemClnEvctTEE	MS0	A or S	<all>									
Y			I	No-Op	Cmp	<none>	No-Op	N/A	N/A	UC	I	UC	The host will only issue these requests from E-state or from S-state. Requester should use these commands if TE State is known.
N				Snptype*									
N		EMS-NO	N/A	<all>									
N	MemRdFwd												Not supported (these requests are only used with HDM-D).
	MemWrFwd												

1. Device Tracked Requester Coherency State (see [Section 11.5.4.11.3](#) for additional details).
2. Cmp-M response is dis-allowed from target if TE mismatch is detected.
3. Target must not downgrade to S-state on TE mismatch if the current state tracked is A-state.
4. NXM case exception. See [Section 3.3.11](#) for details.

C.1.1 Forward Flows for HDM-D

Table C-4 shows the flows that can generate Mem*Fwd messages on CXL.mem from CXL.cache requests. These flows are triggered when a device issues a D2H Request to an address that is mapped with its own CXL.mem address region. This region is referred to as device-attached memory.

Note: The HDM-D region is intended to be used only for cases in which the device or host does not support 256B Flit mode. In cases where the host, switch, and device supports 256B Flit mode, the HDM-DB must be used.

Table C-4. HDM-D Request Forward Sub-table (Sheet 1 of 2)

Legal	Device Request	Host Response on M2S Req				Final Device State		Description
	D2H Req	M2S Req	MetaField	MetaValue	SnpType	Device Cache	Host State	
Y	RdCurr	MemRdFwd	MS0	A	No-Op	I	A	MemRdFwd will reflect the final cache state of the host and for RdCurr it is recommended that the host not change the cache state.
Y				S			S	
Y				I			I	
N				<all>	Snp*			
N			EMS-NO	N/A	<all>			
N	RdShared	MemRdFwd	MS0	A	No-Op	S		The host must only be in S-state or I-state.
Y				S			S	
Y				I			I	
N				<all>	Snp*			
N			EMS-NO	N/A	<all>			
N	RdAny	MemRdFwd	MS0	A	No-Op	S		The host must be in S-state or I-state.
Y				S			S	
Y				I			I	
N				<all>	Snp*			
N			EMS-NO	N/A	<all>			
N	RdOwn/ RdOwnNoData	MemRdFwd	MS0	A	No-Op	E		The host must be in I-state.
N				S				
Y				I			I	
N				<all>	Snp*			
N			EMS-NO	N/A	<all>			

Table C-4. HDM-D Request Forward Sub-table (Sheet 2 of 2)

Legal	Device Request	Host Response on M2S Req				Final Device State		Description
	D2H Req	M2S Req	MetaField	MetaValue	Snptype	Device Cache	Host State	
N	CLFlush	MemRdFwd	MS0	A	No-Op			
N				S				
Y				I		I	S	The host must indicate invalid, but the device must assume that S-state is possible in the host as part of the CLFlush definition.
N				<all>	Snptype*			
N			EMS-NO	N/A	<all>			
N	WOWrInv/ WOWrInvF	MemWrFwd	MS0	A	No-Op			
N				S				
Y				I		NC	I	The host must be in I-state.
N				<all>	Snptype*			
N			EMS-NO	N/A	<all>			
N	CleanEvict/ DirtyEvict/ CleanEvictNoData	N/A	<all>	<all>	<all>			Messages are not sent to the host for device-attached memory.
N	ItoMWr/ WrCur/ WrInv/ CacheFlushed	None	<all>	<all>	<all>			Standard CXL.cache flows are used for these requests.

C.1.2 BISnp for HDM-DB

Table C-5 shows the flows that the device can generate. These flows are only supported to an address that is mapped in the devices own CXL.mem address region. This region is referred to as device-attached memory.

Table C-5. HDM-DB BISnp Flow (Sheet 1 of 2)

Legal	Device Request	Host Response	Final Device State			Description
	S2M BISnp	M2S BIRsp	Device Cache	Host State ¹	EMD	
Y	BISnpCur/ BISnpCurTEE	BIRspI	I	I	UC	Used when the device is requesting a current copy of the cacheline but is not installing the data into its cache.
Y		BIRspS		S		
Y		BIRspE		A		
N		BIRsp*Blk				Block responses are not supported.
Y	BISnpData/ BISnpDataTEE	BIRspI	E, S, or I	I		Used when the device is requesting Shared or Exclusive state of the cacheline.
Y		BIRspS	S or I	S		
N		BIRspE				
N		BIRsp*Blk				
Y	BISnpInv/ BISnpInvTEE	BIRspI	E, S, or I	I		Used when device is requesting Exclusive state of the cacheline including capacity evictions for device Inclusive Snoop Filter evictions. The device may choose to install only in its cache in S-state for device-specific reasons.
N		BIRspS				
N		BIRspE				
N		BIRsp*Blk				
Y	BISnpCurBlk/ BISnpCurBlkTEE	BIRspI	I	I		Used when device is requesting a current copy of a multiple line block but is not installing into its cache. Response is per cacheline.
Y		BIRspS		S		
Y		BIRspE		A		
Y		BIRspIBlk		I — Blk		Used when device is requesting current copy of multiple line block but is not installing into its cache. Response is for the entire block.
Y		BIRspSBlk		S — Blk		
Y		BIRspEBlk		A — Blk		

Table C-5. HDM-DB BISnp Flow (Sheet 2 of 2)

Legal	Device Request	Host Response	Final Device State			Description
	S2M BISnp	M2S BIRsp	Device Cache	Host State ¹	EMD	
Y	BISnpDataBlk/ BISnpDataBlkTEE	BIRspI	E, S, or I	I	UC	Used when device is requesting a Shared or Exclusive copy of a multiple cacheline block. Response is per cacheline.
Y		BIRspS	S or I	S		
N		BIRspE				
Y		BIRspIBlk	E, S, or I	I — Blk		Used when device is requesting a Shared or Exclusive copy of a multiple cacheline block. Response is for the entire block.
Y		BIRspSBlk	S or I	S — Blk		
N		BIRspEBlk				
Y	BISnpInvBlk/ BISnpInvBlkTEE	BIRspI	E, S, or I	I		Used when device is requesting an Exclusive copy of a multiple cacheline block. Response is per line.
N		BIRspS				
N		BIRspE				
Y		BIRspIBlk	E, S, or I	I — Blk		Used when device is requesting an Exclusive copy of a multiple cacheline block. Response is for the entire block.
N		BIRspSBlk				
N		BIRspEBlk				

1. Host State (aka Bias State) tracking in the device can be implemented with a full Table or inclusive SF.

C.2 HDM-H Requests

HDM-H is used only by Type 3 devices for simple memory expansion. Requests from the host to this region always encode SnpType as No-Op and the host can also make arbitrary use of the MetaValue field to store 2-bit encodings, which the device should not interpret. Table C-6 captures the flows for these devices.

Table C-6. HDM-H Memory Request (Sheet 1 of 2)

Legal	Host Request				Device Response					Device State	Description
	M2S Req	MetaField	MetaValue	SnpType	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Metadata/ EMD	
N	<all>	<all>	<all>	Snp*							Snoop encodings are never sent to the HDM-H address region.
Y	MemRd/ MemRdTEE ¹	MS0	<any 2-bit value>	No-Op	<none>	MemData/ MemDataTEE ¹ / MemData-NXM ²	<any>	<any>	N/A	MetaValue	Read that is requesting Meta State field updates to new values.
Y		No-Op	N/A					N/A	UC	Read that does not expect a Meta State field update.	
O-2		EMS	N/A			MemData/ MemDataTEE ¹	EMS-NO	N/A	EMV-NA	UC	EMS in DRS indicates EMV trailer is returned. No Metadata is changed as a result of this message.
O-2						MemData-NXM ²	No-Op	N/A	N/A	UC	NXM response must only return No-Op MetaField for decode errors of requests with MetaField of EMS.
Y		MS0	<any 2-bit value>		Cmp	<none>	<any>	<any>		MetaValue	The host wants to read the Meta State field and update it.
Y	MemInv/ MemInvNT	No-Op	N/A					UC		The host wants to read the Meta State field but does not want to update it.	
N		MS0 or EMS									
Y	MemRdData/ MemRdDataTEE ¹	No-Op	N/A	No-Op	<none>	MemData/ MemDataTEE ¹ / MemData-NXM ²	MS0	I or A	N/A	A	Used for implicit directory state updates in the HDM-H address region. This is the only case in which devices with HDM-H must decode the MetaValue field.
Y							MS0	S		S	
Y								No-Op		N/A	N/A
N		MS0 or EMS	<all>	<all>							
N	MemSpecRd/ MemSpecRdTEE ¹	No-Op	N/A	Snp*							
O-3				No-Op	N/A	No-Op	<none>	<none>	<none>	<none>	N/A

Table C-6. HDM-H Memory Request (Sheet 2 of 2)

Legal	Host Request				Device Response					Device State	Description
	M2S Req	MetaField	MetaValue	Snptype	S2M NDR	S2M DRS	MetaField	MetaValue	DRS Trailer	Metadata/EMD	
N	MemClnEvct										These messages are not used for the HDM-H address region.
N	MemRdFwd										
N	MemWrFwd										
O-4	TEUpdate	MS0	00b (clear TE)	<Length>	Cmp	<none>	No-Op	N/A	N/A	UC	Commands used to set or clear TE State for region of memory. The TE is set when MetaValue is 01b and cleared when MetaValue is 00b. This command is optionally supported by devices that support TSP.
O-4			01b (set TE)								
N			10b or 11b								
N		No-Op or EMS	<all>								

1. TEE opcodes are optionally enabled.
2. Returned when the Switch or Device cannot positively decode that the address is HDM-H vs. HDM-D* address region. This response allows the host to complete the request by converging the flows for this Decode Miss error between the HDM-H and HDM-D*. See [Section 3.3.11](#) for details.

C.3 HDM-D RwD

Table C-7 captures the Request with Data (RwD) flows for HDM-D address regions used by Type 2 devices. EMD is not applicable for HDM-D.

Table C-7. HDM-D Memory RwD

Legal	Host Request					Device Response				Device State		Description
	M2S RwD	MetaField	MetaValue	SnpType	RwD Trailer	S2M NDR	S2M DRS	MetaField	MetaValue	Dev Cache	Host State	
N	MemWr/ MemWrPtl	MS0-EMS	A	Snp*			<none>					Snoop encodings are never sent with A-state because the host must have an exclusive copy of the cacheline.
Y(1)		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	A	The host wants to update memory and keep an exclusive copy of the cacheline.
O-2		EMS			EMV							
N		MS0-EMS	S	Snp*								
Y		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	S	The host wants to update memory and keep a shared copy of the cacheline.
O-2		EMS			EMV							
Y(1)		MS0	I	SnpInv ¹	N/A	Cmp		No-Op	N/A	I	I	The host wants to write the cacheline back to memory and does not retain a cacheable copy. In addition, the host was not granted ownership of the cacheline before performing this write and needs the device to invalidate the device's caches before performing the writeback to memory.
O-2		EMS			EMV							
N		MS0-EMS		SnpData								No use case.
N				SnpCur								
Y(1)		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	I	The host wants to update memory and will end with host caches in I-state. Use is for Dirty (M-state) Cache Evictions in the host.
O-2		EMS			EMV							
N		MS0-EMS	<all>	<all>		Cmp		MS0	<all>	<any>	<any>	The device never returns the Meta State field for a write.
N		No-Op	N/A	<all>								The host must always define the MetaValue field for writes.
N		EMD	E-No-Op	<all>								
N		BISnConflict	<all>	<all>	<all>							
N	MemRdFill/ MemWrTEE/ MemWrPtlTEE/ MemRdFillTEE/ TEUpdate											TEE is not supported for HDM-D in this revision of the specification.

1. Snoopable Writes are not supported in Shared FAM use cases.

C.4 HDM-DB Rwd

Table C-8 captures the Request with Data (Rwd) flows for HDM-DB address regions used by Type 2 devices and Type 3 devices.

Table C-8. HDM-DB Memory Rwd (Sheet 1 of 2)

Legal	Host Request					Device Response				Device State			Description	
	M2S Rwd	MetaField	MetaValue	SnpType	RwD Trailer	S2M NDR	S2M DRS	MetaField	MetaValue	Dev Cache	Host State	EMD ¹		
N	MemWr/ MemWrTEE ² / MemWrPtl/ MemWrPtITEE ²	MS0-EMS	A	Snp*			<none>						Snoop encodings are never sent with A-state because the host must have an exclusive copy of the cacheline.	
Y(1)		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	A	UC	The host wants to update memory and keep an exclusive copy of the cacheline.	
O-2		EMS			EMV							EMV		
N		MS0-EMS	S	Snp*										
Y		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	S	UC	The host wants to update memory and keep a shared copy of the cacheline.	
O-2		EMS			EMV							EMV		
Y(1)		MS0	I	SnpInv ³	N/A	Cmp		No-Op	N/A	I	I	UC	The host wants to write the cacheline back to memory and does not retain a cacheable copy. In addition, the host was not granted ownership of the cacheline before performing this write and needs the device to invalidate the device's caches before performing the writeback to memory.	
O-2		EMS			EMV							EMV		
N		MS0-EMS		SnpData									No use case.	
N		MS0-EMS		SnpCur										
Y(1)		MS0		No-Op	N/A	Cmp		No-Op	N/A	I	I	UC	The host wants to update memory and will end with host caches in I-state. Use is for Dirty (M-state) Cache Evictions in the host.	
O-2		EMS			EMV							EMV		
N		MS0-EMS	<all>	<all>		Cmp		MS0	<all>	<any>	<any>		The device never returns the Meta State field for a write.	
N		No-Op	N/A	<all>										The host must always define the MetaValue field for writes.
N		EMD	E-No-Op	<all>										
Y	BIConflict	No-Op	N/A	No-Op	N/A	BIConflict Ack		No-Op	N/A	UC	UC	UC	Used as part of the BISnp conflict handling flow.	
N				Snp*										
N		MS0-EMS	<all>	<all>										

Table C-8. HDM-DB Memory RwD (Sheet 2 of 2)

Legal	Host Request					Device Response				Device State			Description		
	M2S RwD	MetaField	MetaValue	SnpType	RwD Trailer	S2M NDR	S2M DRS	MetaField	MetaValue	Dev Cache	Host State	EMD ¹			
O-4	MemRdFill/ MemRdFillTEE ²	No-Op	N/A	No-Op	N/A	<none>	MemData/ MemDataTEE ²	<any>	<any>			UC	Support for this command is required by devices that support TSP to support host-side memory encryption. The result of this read does not change meta-state and returns only data to the host. The use case is expected for partial write merging that is not possible in the device with host-side encryption.		
O-4					MemData-NXM										
N						Snp*									
N					MS0	<all>									
O-4	TEUpdate ²	MS0	00b (clear TE)	<Length>	N/A	Cmp	<none>	No-Op	N/A			UC	Commands used to set or clear TE State for region of memory. The TE is set when MetaValue is 01b and cleared when MetaValue is 00b. This command is optionally supported by devices that support TSP.		
O-4			01b (set TE)												
N			10b or 11b												
N		No-Op or EMS	<all>												

1. Applicable only with EMD enabled.
2. TEE opcodes are optionally enabled.
3. Snooperable Writes are not supported in Shared FAM use cases.

C.5 HDM-H Rwd

Table C-9 captures the Request with Data (Rwd) flows for the HDM-H memory region.

Table C-9. HDM-H Memory Rwd

Legal	Host Request					Device Response				Device State	Description
	M2S Rwd	MetaField	MetaValue	Snptype	Rwd Trailer	S2M NDR	S2M DRS	MetaField	MetaValue	Metadata/EMD ¹	
N	MemWr/ MemWrPtl/ MemWrTEE ² / MemWrPtlTEE ²	MS0-EMS	<all>	Snptype*			<none>				Snptype encodings are never sent to the HDM-H.
Y		MS0	<any 2-bit value>	No-Op	N/A	Cmp CmpTEE ²		No-Op	N/A	MetaValue	The host wants to update memory. The host sets a value in the MetaValue field. The device optionally stores that MetaValue field value. Note: MemWrPtl is not expected to be used with host-side encryption.
O-2		EMS	N/A		EMV					EMV	
N				No-Op		Cmp		MS0	<any>		The host never needs the Meta State field value returned from a write.
N		No-Op	N/A	<all>							The host always sends MS0 to avoid the need for a Read-modify-write in the device for the MetaValue field.
N	BIConglict	<all>	<all>	<all>							This message is not used for the HDM-H memory region.
O-4	MemRdFill/ MemRdFillTEE ²	No-Op	N/A	No-Op	N/A	<none>	MemData/ MemDataTEE ²	<any>	<any>	UC	Support for this command is required by devices that support TSP to support host-side memory encryption. The result of this read does not change meta-state and returns only data to the host. The use case is expected for partial write merging that is not possible in the device with host-side encryption.
O-4							MemData-NXM				
N				Snptype*							
N		MS0	<all>								

1. EMD is applicable only if enabled.
2. TEE opcodes are optionally enabled.